Multithreaded Programming
— Chapter 4

叶冠华

g.ye@bupt.edu.cn

School of Computer Science (National Pilot Software Engineering School)

北京郵電大學

Chapter 4 Multithreaded Programming

CHAPTER OBJECTIVES

- To introduce the notion of a thread—a fundamental unit of CPU utilization that forms the basis of multithreaded computer systems.
- To discuss the APIs for the Pthreads, Windows, and Java thread libraries.
- To explore several strategies that provide implicit threading.
- To examine issues related to multithreaded programming.
- To cover operating system support for threads in Windows and Linux.

- I. 4.1 Overview
  - why thread needed: motivation, benifits, composition
- II. 4.2 Hardware/software for Multicore programming (parallel programming)
  - hardware: multicore/multiprocessor system
  - software programming modes: types of parallelism
- III. 4.3 Multithread programming model
  - thread types
    - kernel thread , e.g. in Windows
    - user thread , e.g. 协程，纤程
  - mapping from user thread to kernel thread, to allocate CPU to user threads
    - one-to-one
    - many-to-one
    - many-to-many
- IV. Multithread programming schemes/supporting
  - 4.4 Thread libraries (API)
    - Pthread
    - Windows threads
    - Java threads
  - 4.5 Implicit threading (creation and management of threading by compiler or run-time libraries)
    - Thread pools
    - openMP
    - grand central dispatch
- V. 4.6 Threading issues (in designing multithreading programs)
  - fork() and exec()
  - thread cancellation
  - signal handling
  - thread local storage (TLS)
  - scheduler activations
- VI. Linux as Case Study-4-Threads in Linux
- VIII. Linux experiments: 1. Threads management and communications; 2. 多核多线程编程及性能对比分析
- IX. Examples and exercises
- VII. Appendix
  - Appendix 4-1 HyperThread(HT)
  - Appendix 4-2 多核/众核CPU
  - Appendix 4-3 多核/HT speedup
  - Appendix 4-4 基于分治的问题求解复杂性分析
  - Appendix 4-5 算法改进与硬件迭代

Terminology-第4章

concurrency, parallelism	并发, 并行
heavyweight process HWP, lightweight process LWP	重质进程, 轻质进程
coroutine, threadFiber	协程, 纤程
bound	绑定
thread pool	线程池
signal	信号
thread-Local Storage	线程本地存储

Multithreaded Programming(多线程化程序设计/编程)

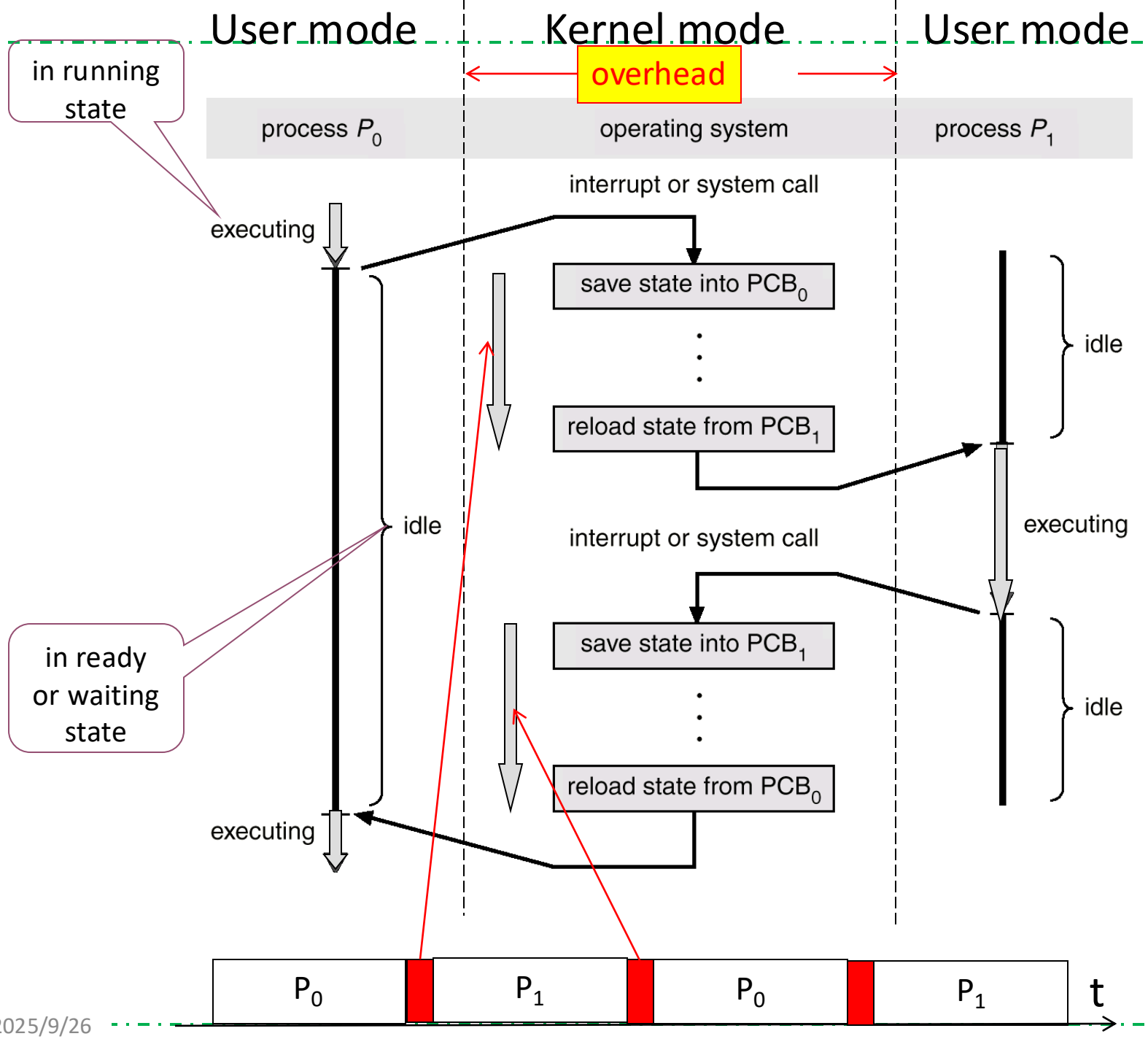
Why thread and multithreaded programming needed ?

- Thread
  - in multiprogramming systems, to reduce the overhead of process context switch in kernel mode, and increase CPU utilization/利用率, refer to next slide
- Multithreaded
  - in multicore, multiple processor and distributed systems/platforms, to support parallel/concurrent programming
    - so as to speedup computation and improve throughput/吞吐率

7th edition

Threads

The process model introduced in Chapter 3 assumed that a process was an executing program with a single thread of control. Most modern operating systems now provide features enabling a process to contain multiple threads of control. This chapter introduces many concepts associated with multithreaded computer systems, including a discussion of the APIs for the Pthreads, Win32, and Java thread libraries. We look at many issues related to multithreaded programming and how it affects the design of operating systems. Finally, we explore how the Windows XP and Linux operating systems support threads at the kernel level.



Process context switch with higher overhead

Process → Thread:

to improve CPU utilization, and support higher degree of multiprogramming

(1) 减少CPU switch 时间

(2)快速启动程序（分配资源、创建线程）

课程思政

课程思政

4.1 Overviews

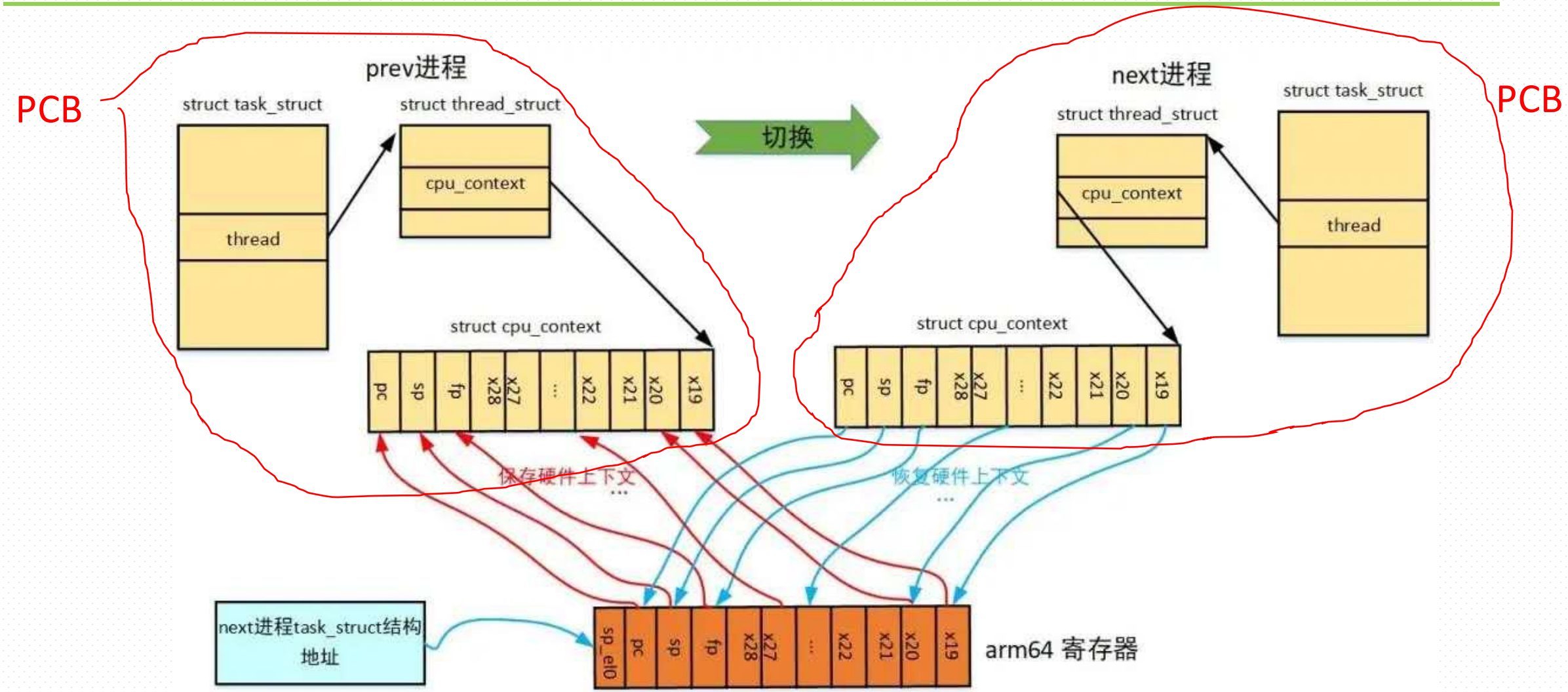
4.1.1 Why thread needed

- As described in Chapter 3, the process is as the
  - unit of resource allocation
  - unit of CPU scheduling, running on CPU

- The costs of process management in kernel mode, such as costs of process creation, cancellation, process context switch due to, e.g. CPU scheduling, and process communication, are somewhat high, refer to Fig.3.4, ▶ and the concurrency/并发 (or multiprogramming degree) may be limited

- To reduce costs of process management and improve degrees of concurrency and parallelism/并行, the thread is presented

Linux/ARM平台下进程切换—process context switch



硬件上下文切换

（Intel x86 平台）进程上下文切换开销包括

直接开销

- 1. 切换页表全局目录
- 2. 切换内核态堆栈
- 3. 切换硬件上下文（进程恢复前，必须装入寄存器的数据统称为硬件上下文）
  - ip(instruction pointer)：指向当前执行指令的下一条指令
  - bp(base pointer): 用于存放执行中的函数对应的栈帧的栈底地址
  - sp(stack poinger): 用于存放执行中的函数对应的栈帧的栈顶地址
  - cr3:页目录基址寄存器，保存页目录表的物理地址
- 4. 刷新TLB
- 5. 系统调度器的代码执行

间接开销-降低CPU访存速度

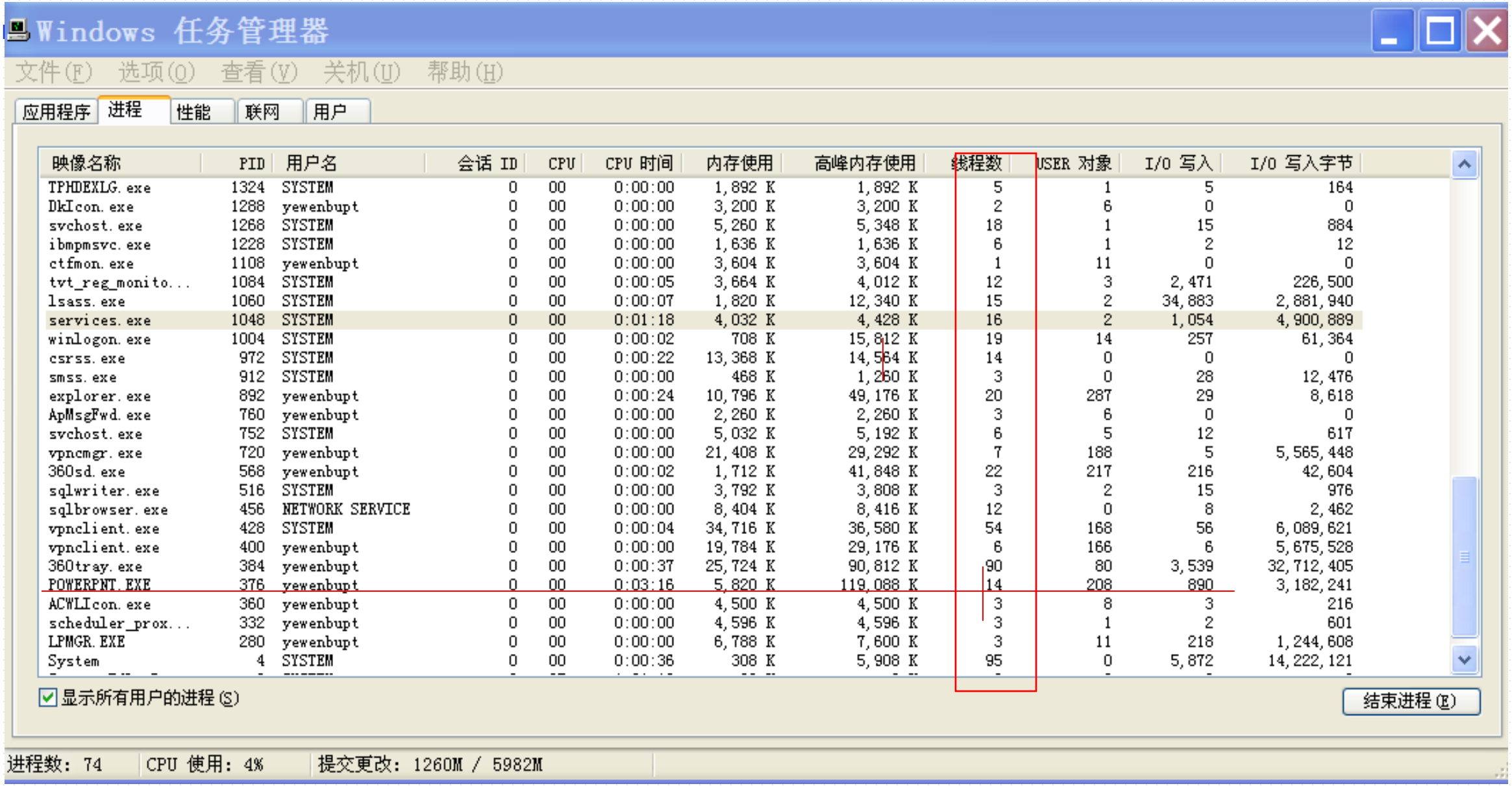
- 切换到一个新进程后，由于缓存cache并不热，CPU速度运行会慢一些。如果进程始终都在一个CPU上调度还好一些，如果跨CPU的话，之前热起来的TLB、L1、L2、L3因为运行的进程已经变了，所以以局部性原理cache起来的代码、数据将失效，导致新进程穿透到内存的IO会变多

Why thread needed

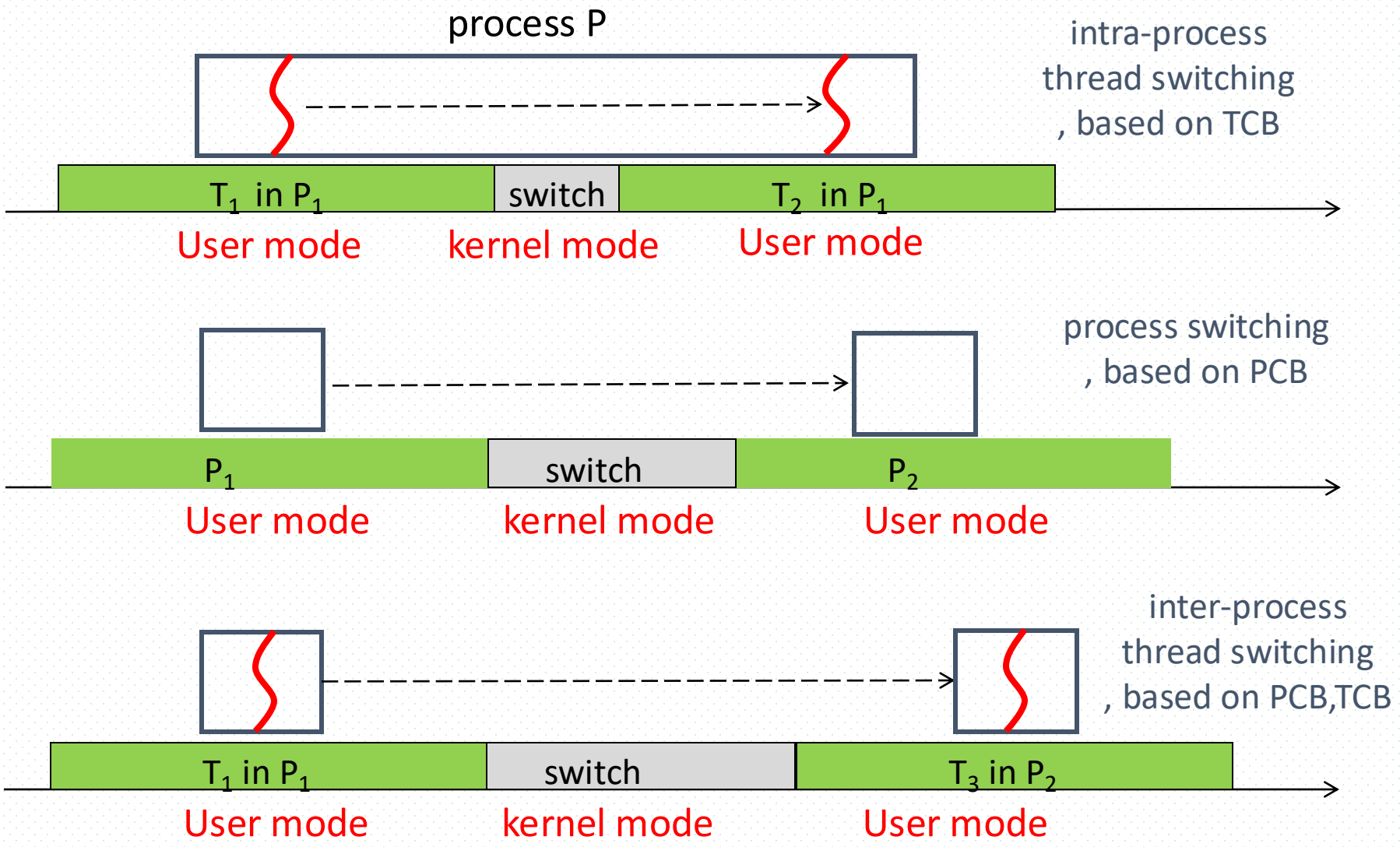
- In thread-supporting OS, a process may contain several threads,
  - the process is as the unit of resource allocation
  - while the threads as the unit of CPU scheduling and the running entity on CPU
- The threads in the same process share the resources allocated to the process

- Merits
  - less costs of context switching among the threads in the same process than that of process context switching
  - improving concurrency and parallelism, less possibility of being blocked
  - parallel executing of programs, higher computation speed /* 程序并行执行
    - 多处理器系统，线程被内核独立调度，可在多个处理器或CPU核上运行
    - 当一个进程中含有多个线程时，进程可分得多个CPU或CPU核，实现程序并行执行

More than one thread in a process



Intra-process vs inter-process thread switching



Example: Go协程（用户级线程）测试

表4-1 特定硬件平台下的切换/系统调用/中断测试时间

类型	系统调用	进程切换	Linux（伪）线程切换	软中断(trap)	Go协程切换(ULT切换)
耗费时间	200ns	3.5us (2.7-5.48us)	3.8us	3.4us	120ns

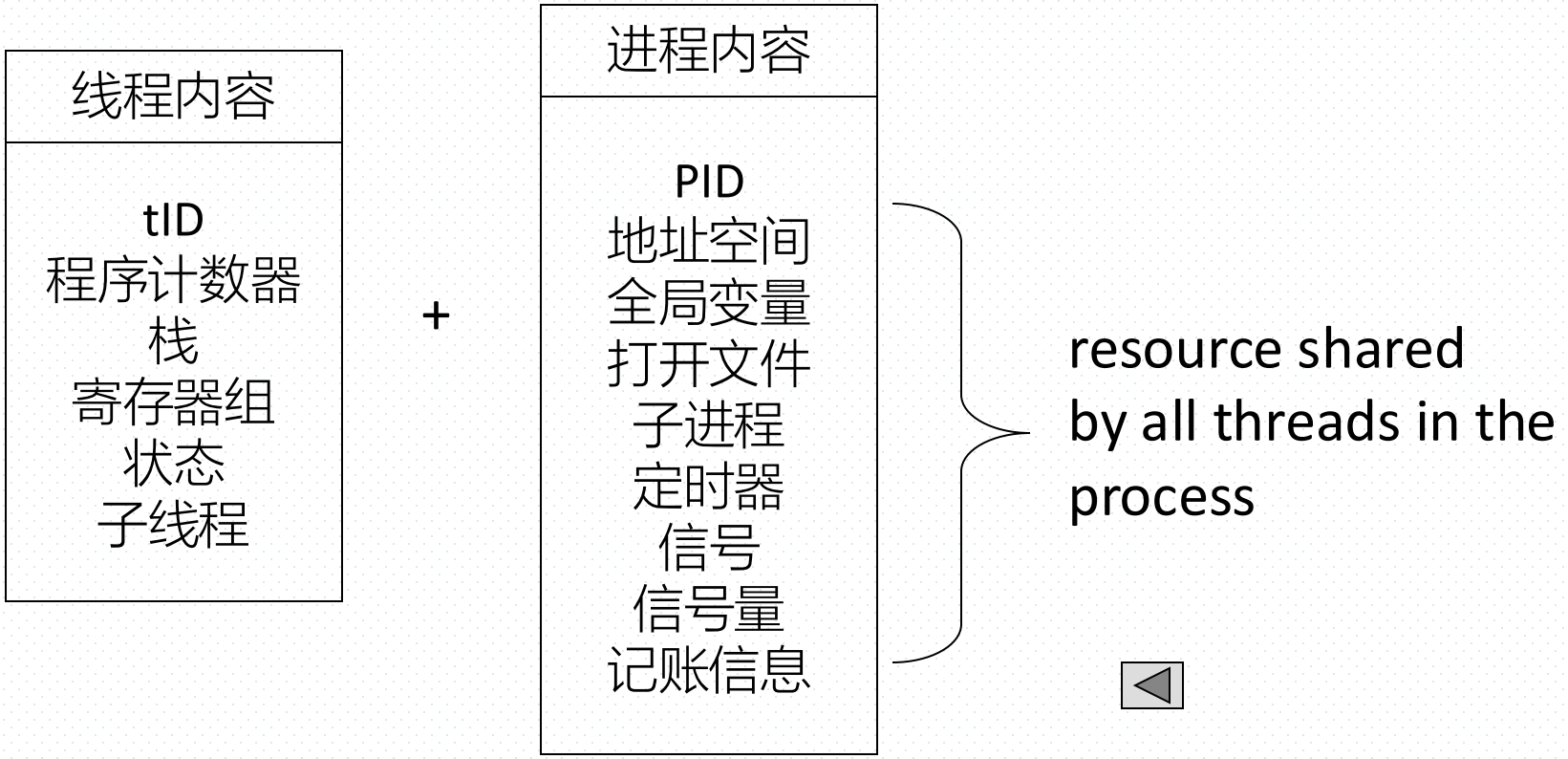
- 协程切换的开销120ns，进程切换开销大约3.5us，大约是其的三十分之一；也低于系统调用开销
- 协程内存开销
  - 初始化创建协程时，OS为其分配2KB的栈。而内核级线程栈（Linux线程？）要比2KB大的多，可以通过ulimit 命令查看，如10M；
  - 如果对每个用户创建一个协程去处理，100万并发用户请求只需要2G内存；但如果用（内核）线程模型，则需要10T内存

Thread Concept

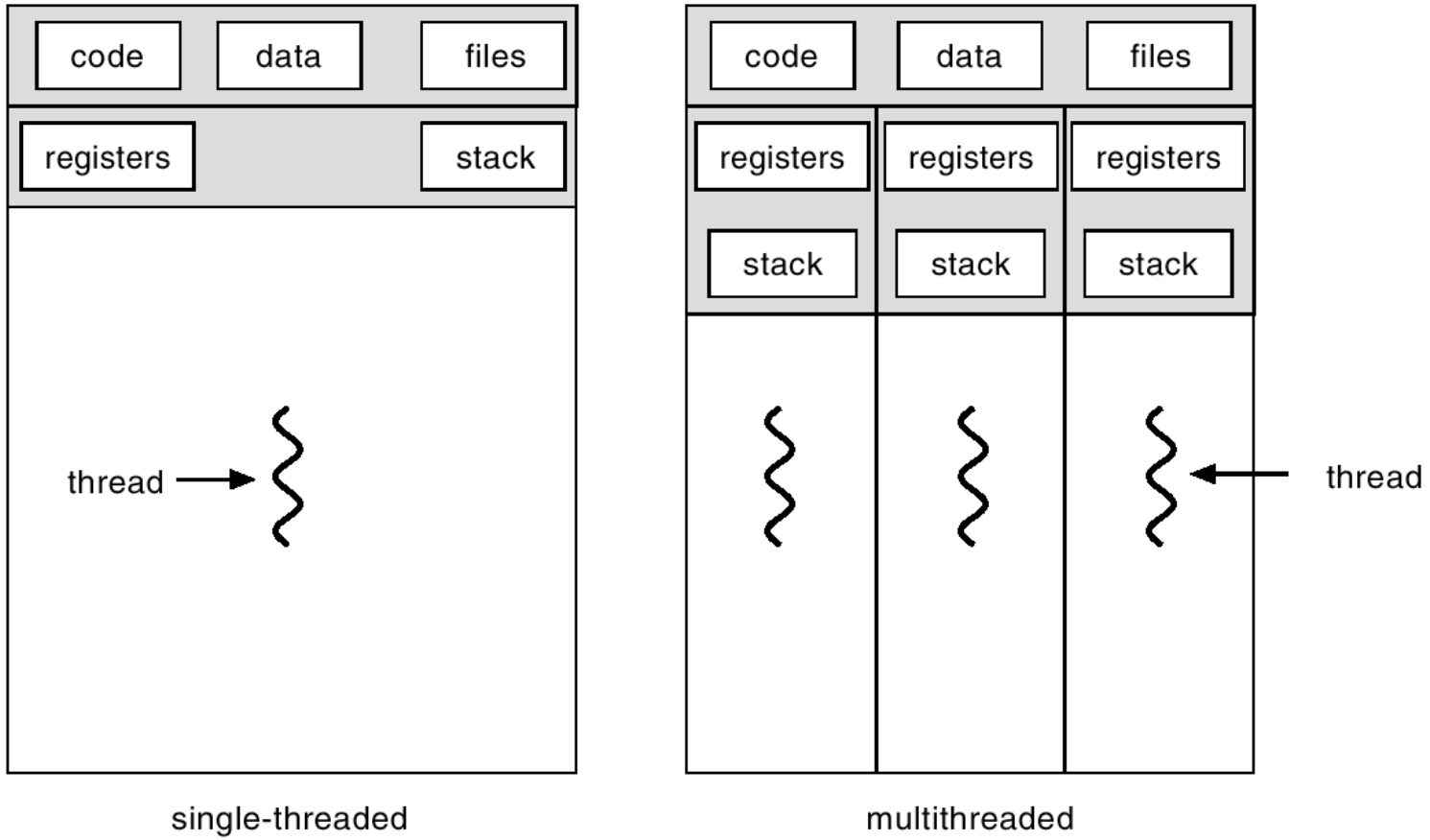
- Thread Definition
  - a basic unit/entity of CPU utilization, that is, the basic unit /entity for CPU scheduling
  - as a part of a process and can indecently runs on CPU
- A process may contain several threads. As a basic unit for CPU scheduling, a thread in a process shares with other threads that belongs to the same process its code section, data section and other resources
  - resources are allocated to the process, and all threads in the process share these resources
  - tread = thread ID + program counter + a register set + stack
    — refer to Fig.4.1 and the figure in the next slide

- Traditional process, also named heavyweight process(重质进程, HWP), has a single thread of control
- A thread is also called a lightweight process (轻质进程, LWP)

Thread Concept



Single and Multithreaded Processes

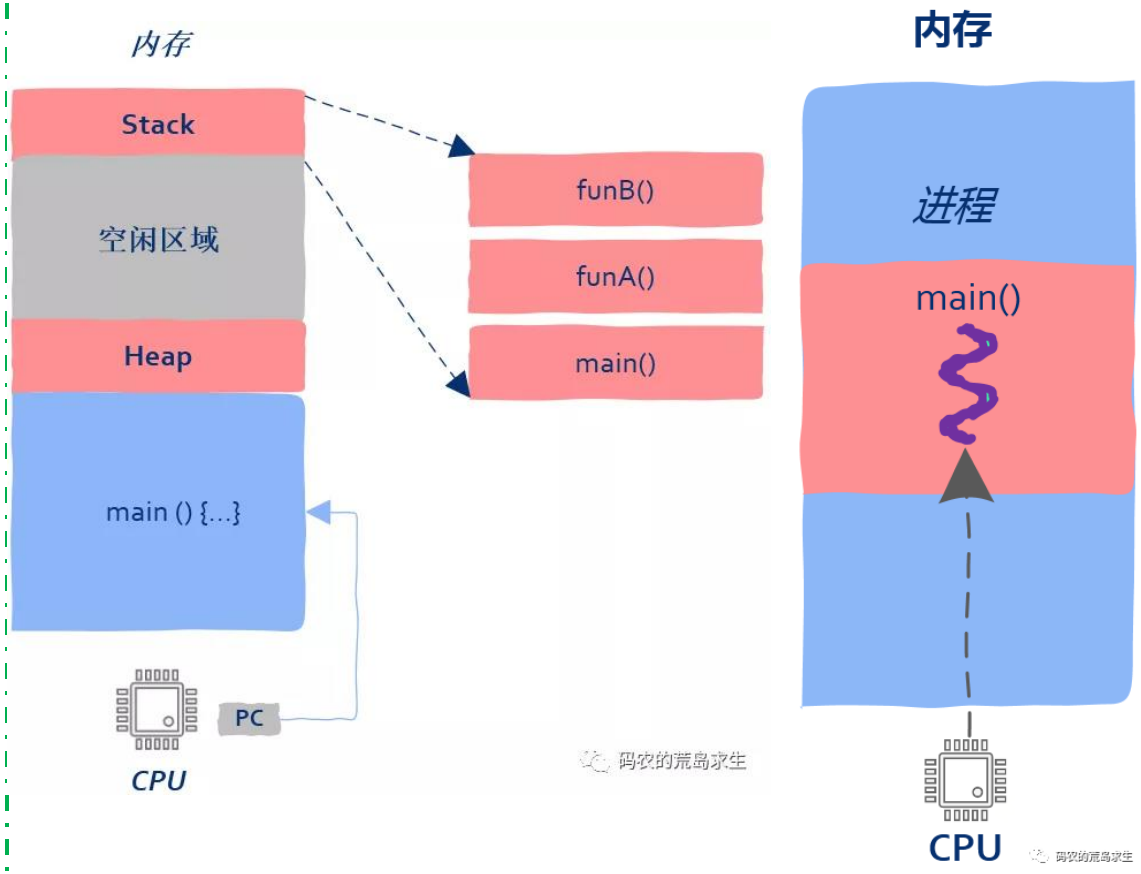


a traditional process as a single-thread

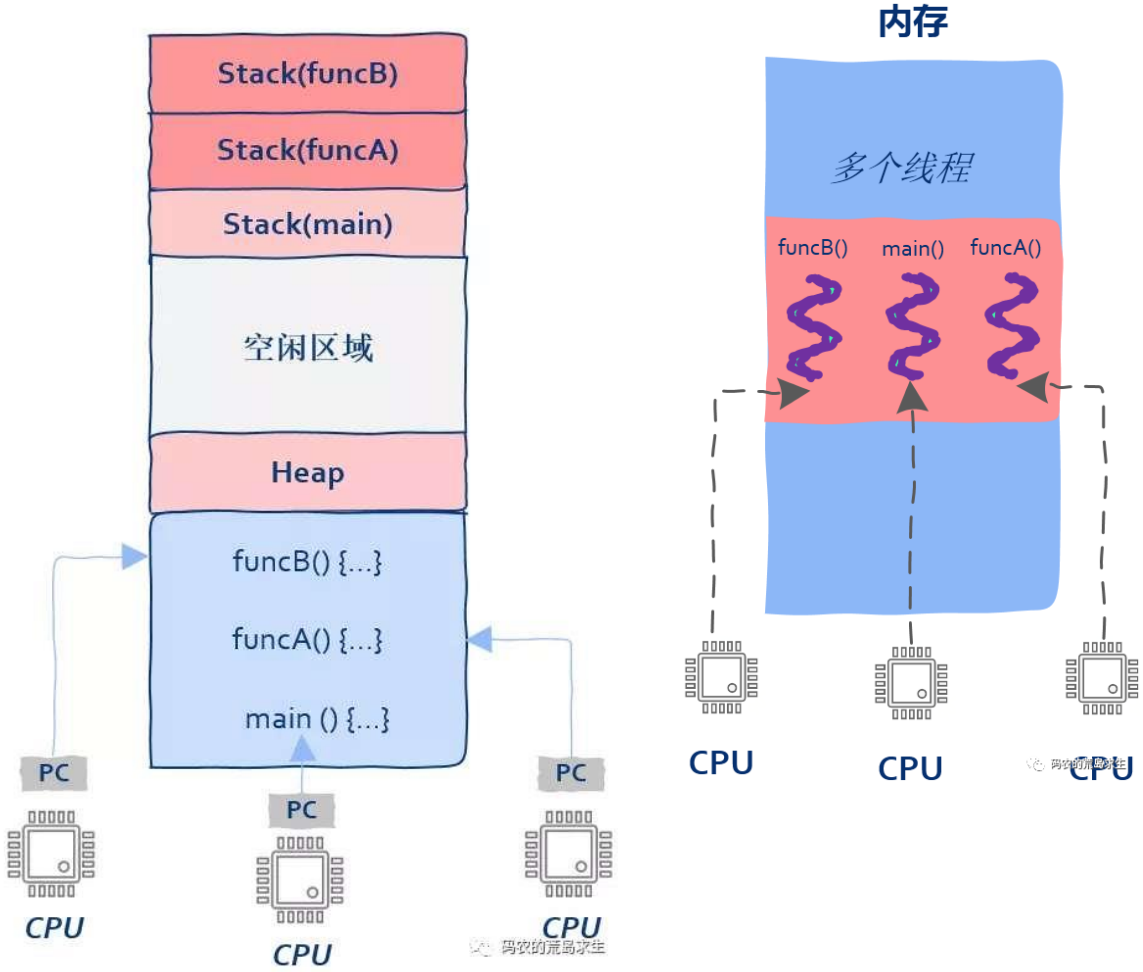
more than one thread in one process

Fig.4.1 Single and Multithreaded Processes

process / single- thread



multiple threads in a process



Overviews

- Example:　web server program
  - multithreaded web server process，海量高并发处理
  - more than one thread in the process
  - each thread serves for a client access request, many clients can be served concurrently

- Benefits
  - Responsiveness
  - resource sharing
  - Economy
  - utilization of multiple-processor architectures
  - Refer to Appendix 4-1 关于线程 for more details about threads

4.2 Multicore Programming

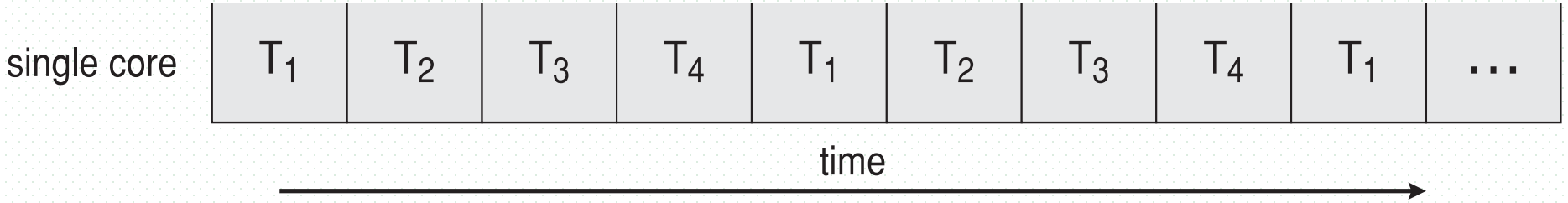
- Parallel programming on multicore or multiple processor systems
- **Multicore** or **multiprocessor** systems putting pressure on programmers, challenges include:
  - dividing activities
    - 分治递归，算法设计与分析
  - balance
  - data splitting
  - data dependency
  - testing and debugging

并行 vs 并发 implies a system can perform more than one task simultaneously

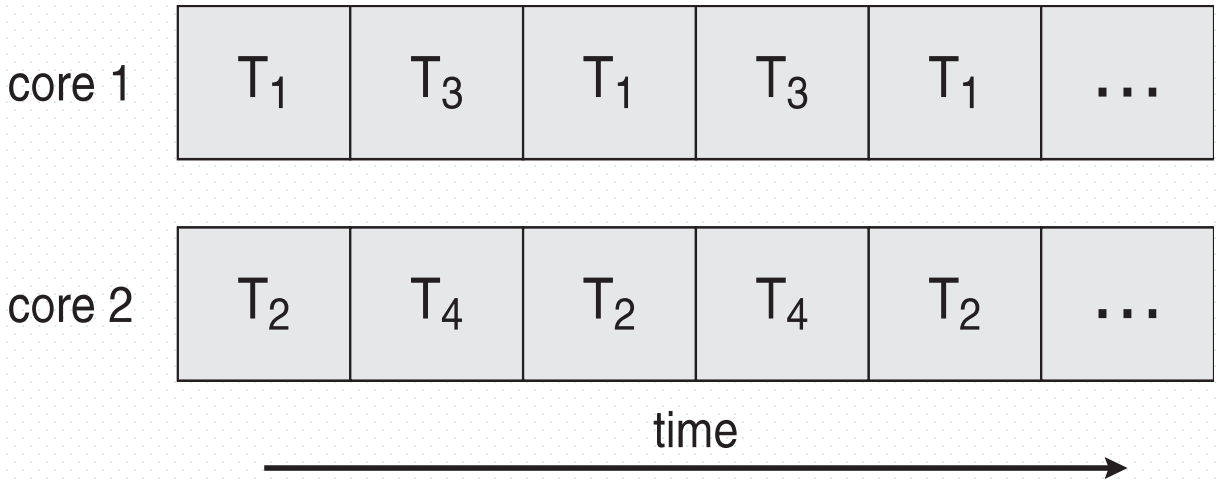
- ***Concurrency*** supports more than one task making progress
  - single processor / core, scheduler providing concurrency
    - 微观串行，宏观并行

Concurrency vs Parallelism

- Concurrent execution of several threads on single-core system:



- Parallelism on a multi-core system:



Multicore Programming (Cont.)

- Two types of parallelism
- **Data parallelism**
  - distributes subsets of the same data across multiple cores, same operation on each
  - e.g. , summing the contents of an array of size N, A[0, N-1]
    - on a single-core system, one thread would simply sum the elements [0] ...[N − 1].
    - on a dual-core system, however, thread A, running on core 0, could sum the elements [0] ...[N/2 − 1] while thread B, running on core 1, could sum the elements [N/2] ...[N − 1]. The two threads would be running in parallel on separate computing cores
- **Task parallelism**
  - distributing threads across cores, each thread performing unique operation, Each thread is performing a unique operation.
  - different threads may be operating on the same data, or they may be operatingon different data
  - e.g. thread A calculates the average of the elements of array A[0, N-1], and thread B find the largest element in array A[0, N-1]

4 .3 Multithread Programming Models

- The threads can be identified as
  - user threads, or user-level threads (ULT),
    - e.g. coroutine协程
    - management (creating, scheduling,..) is conducted by the user-level threads library, or by programming environments
    - OS may be still process-based
  - kernel threads, or kernel-level threads (KLT)
    - OS is responsible for managing threads

- Three primary thread libraries
  - POSIX Pthreads
  - Win32 threads
  - Java threads

用户开发1个多线程C++程序P={threads}，运行在多CPU/多核系统上，
OS如何为这些线程分配CPU？

Multithread Programming Models

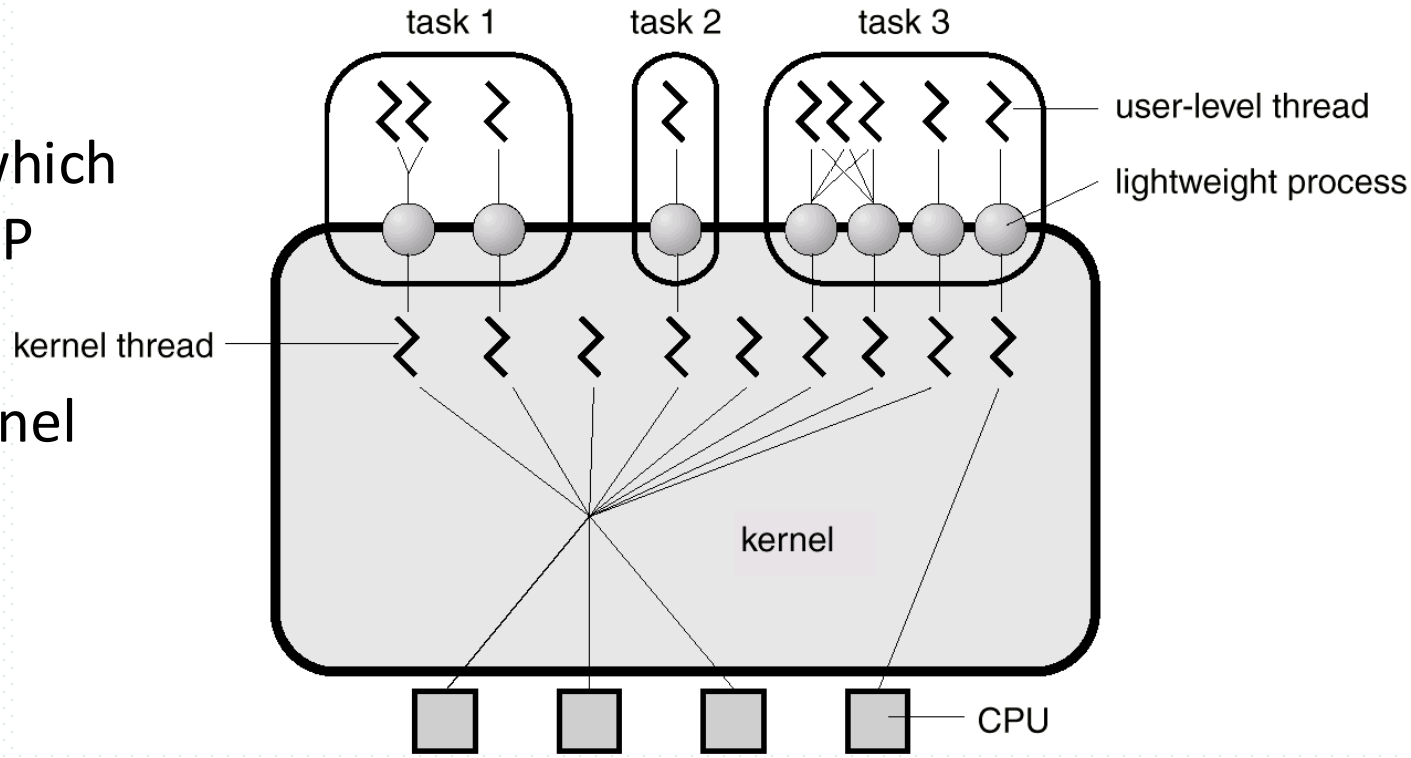
- In the system (e.g. Windows 10) supporting kernel threads, OS scheduler assigns CPU to kernel threads. For the user-level threads, how to occupy the CPU ?

- How to implement ULTs by KLTs
  - mapping between user threads and kernel threads
  - mapping is conducted by the compilers

Thread Scheduling

- Kernel-level threads are managed by OS, while user-level threads by thread library
- How to conduct scheduling involving KLT and ULT

- Two-level scheduling for ULT
  - local scheduling
    - how the threads library decides which thread to put onto an available LWP
  - global scheduling
    - how the kernel decides which kernel thread to run next



- Refer to Fig.4.0.3 and following figures

User-thread-to- Kernel-thread Mapping

- Many-to-one
- One-to-one
- Many-to-many

-

Many-to-one

- Many user-level threads mapped to single kernel thread/process.
- Example
  - GNU Portable Threads
  - Solaris Green threads
- Demerit
  - one thread blocking causes all to block
  - multiple threads may not run in parallel on muticore system because only one may be in kernel at a time
- Few systems currently use this model

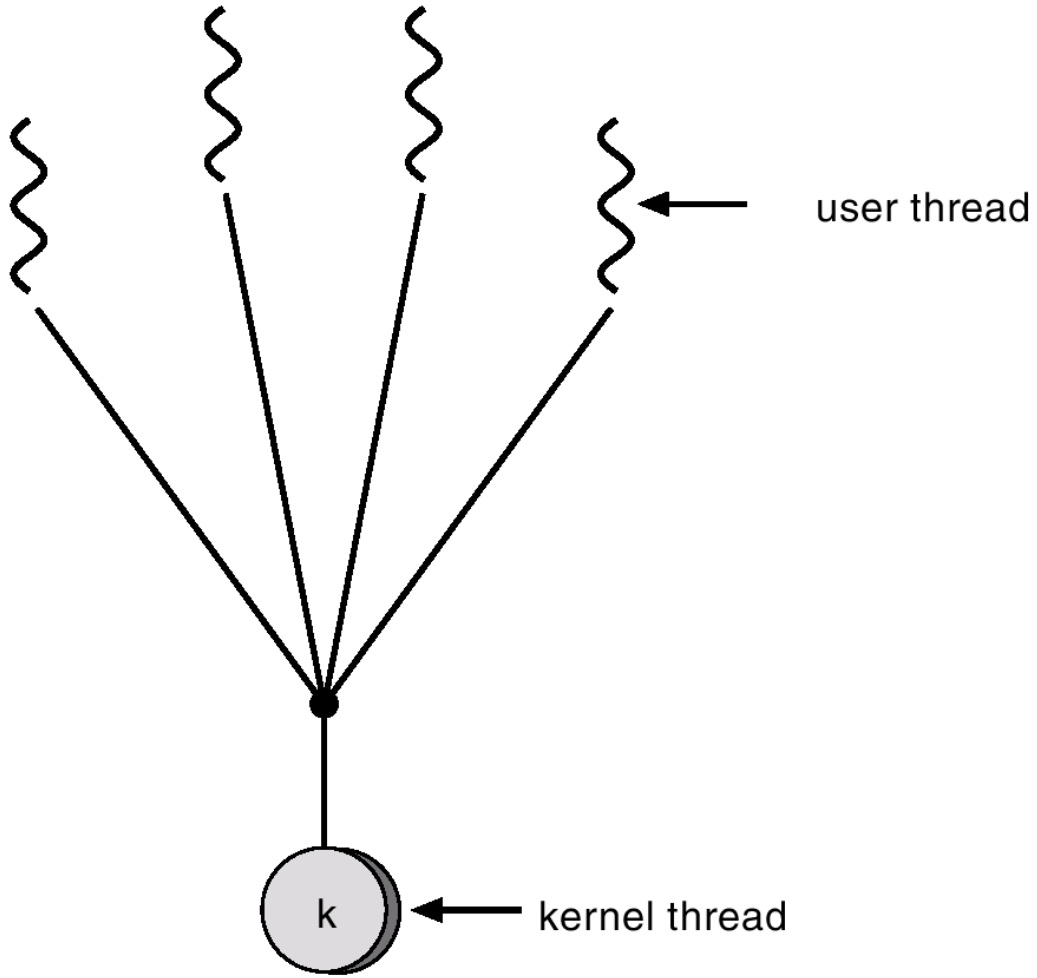


Fig.4.2 Many-to-One Model

One-to-one Mapping

- Each user-level thread maps to kernel thread
  - creating a user-level thread creates a kernel thread

- More concurrency than many-to-one
- However, the number of threads per process sometimes restricted due to overhead

- Examples
  - Windows
  - Linux
  - Solaris 9 and later

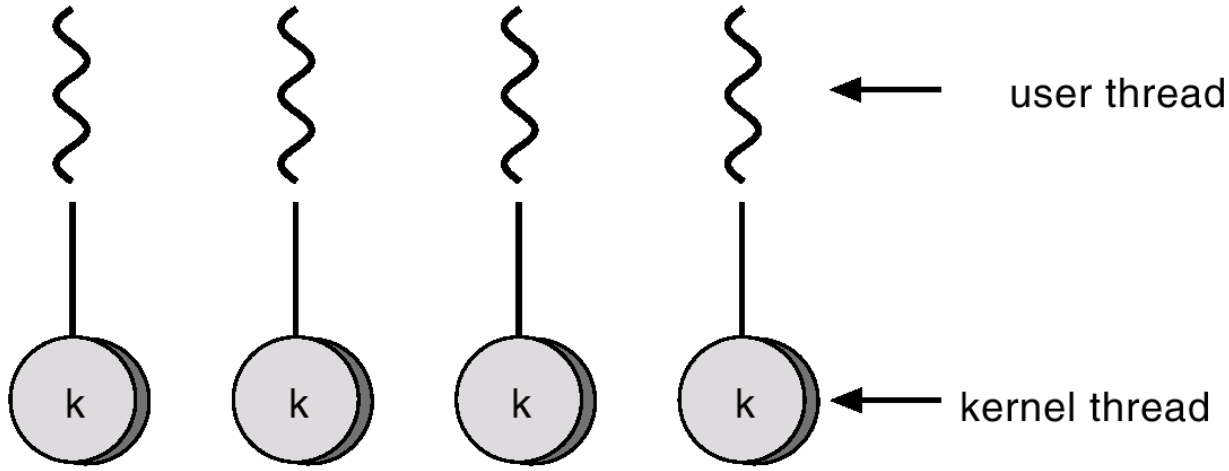


Fig.4.3 One-to-One Model

Many-to-many Mapping

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- E.g.
  - Solaris prior to version 9
  - Windows with the ThreadFiber (纤程) package

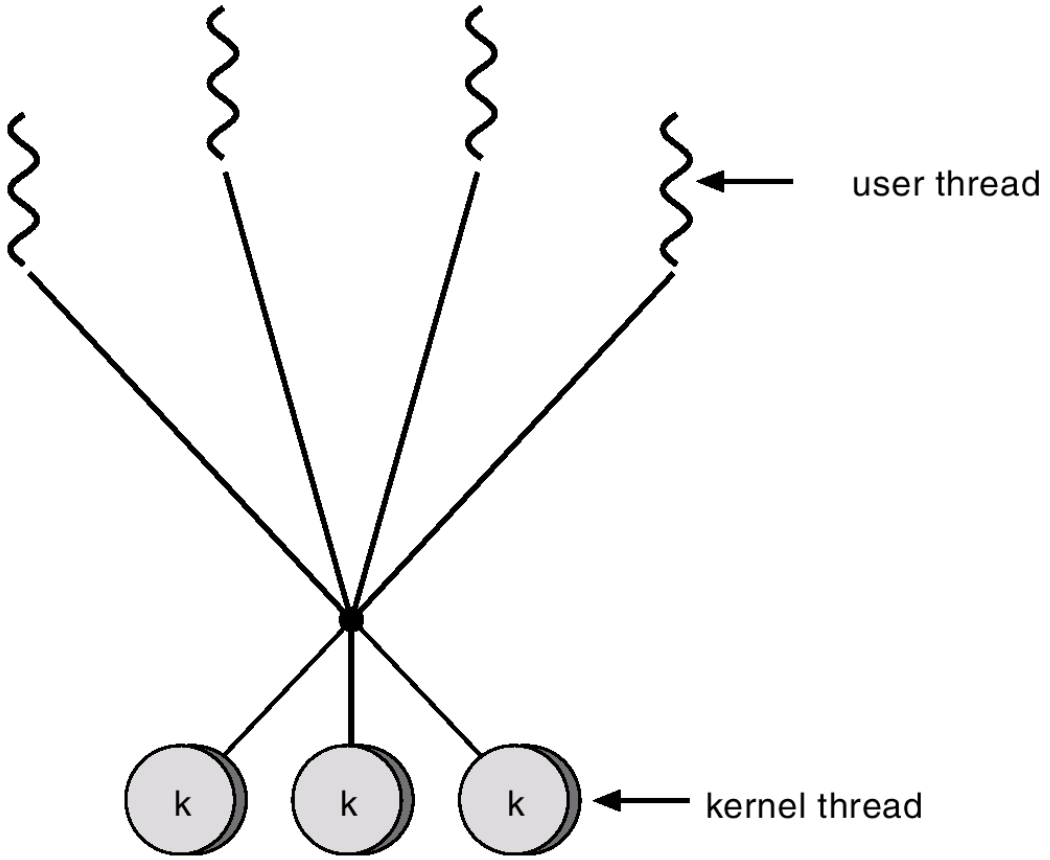
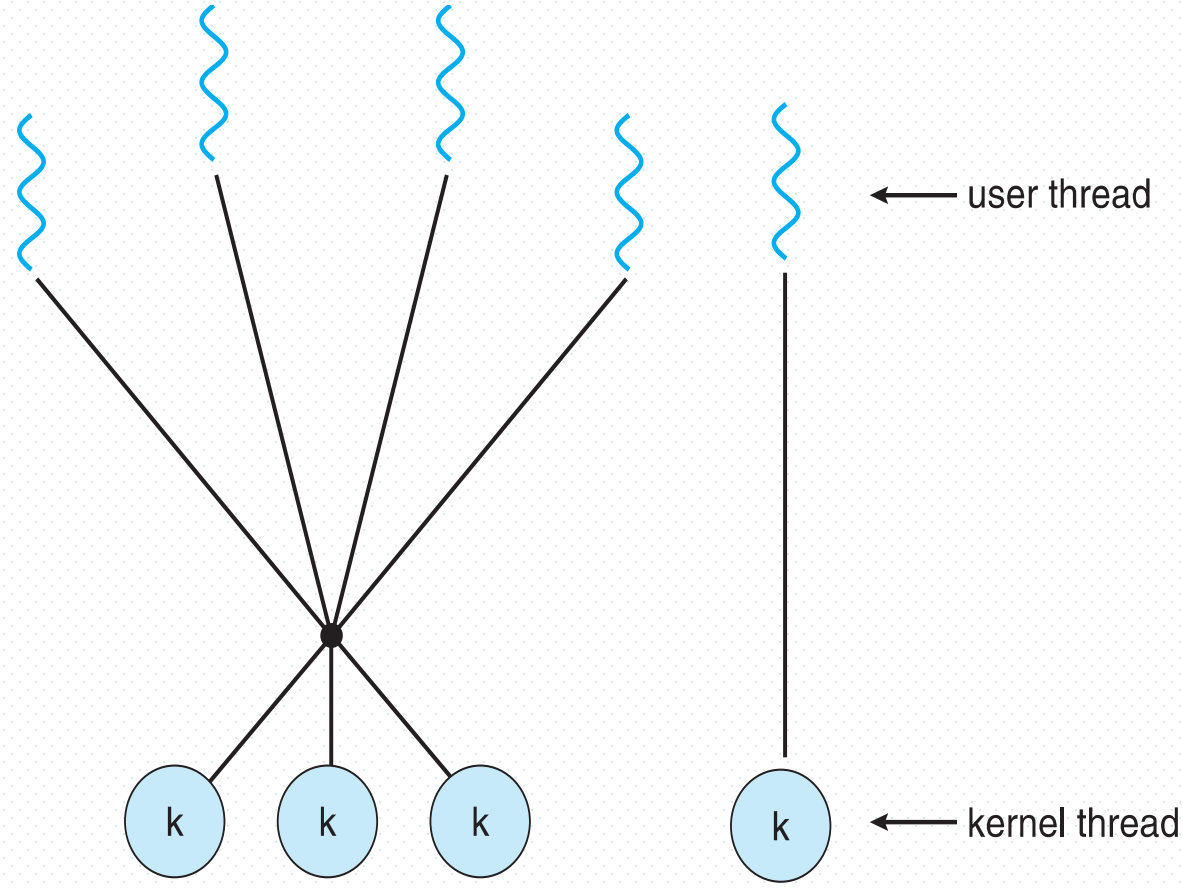


Fig.4.4 Many -to-many Model

Two-level Model

- Similar to M:M, except that it allows a user thread to be **bound/绑定** to kernel thread
  - 将用户级线程指派到特定的内核级线程
  - 类似于多核/多处理器系统中，CPU调度中的CPU亲和
    - 将进程/线程固定分配到某些CPU核
- Examples
  - IRIX
  - HP-UX
  - Tru64 UNIX
  - Solaris 8 and earlier



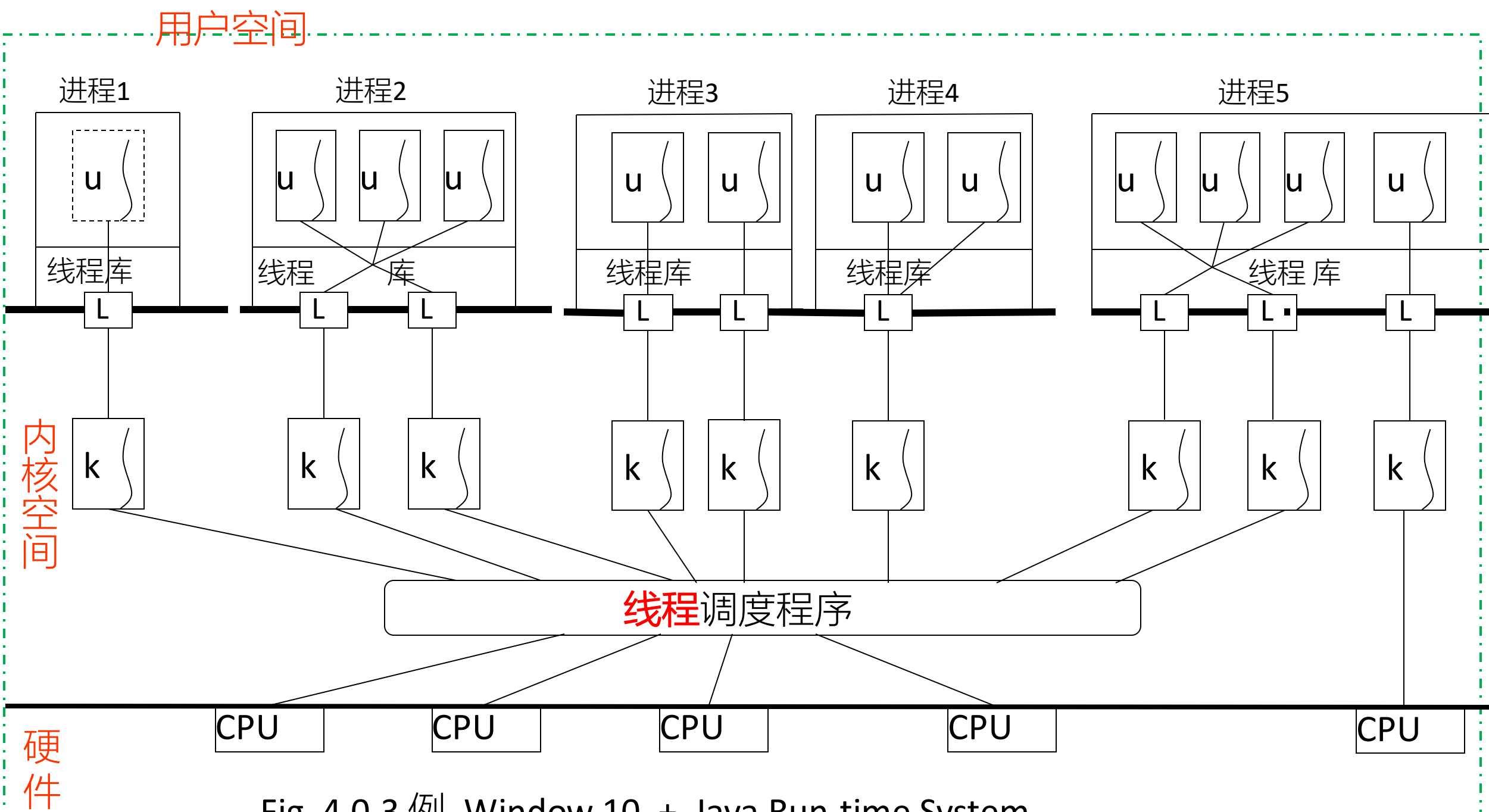
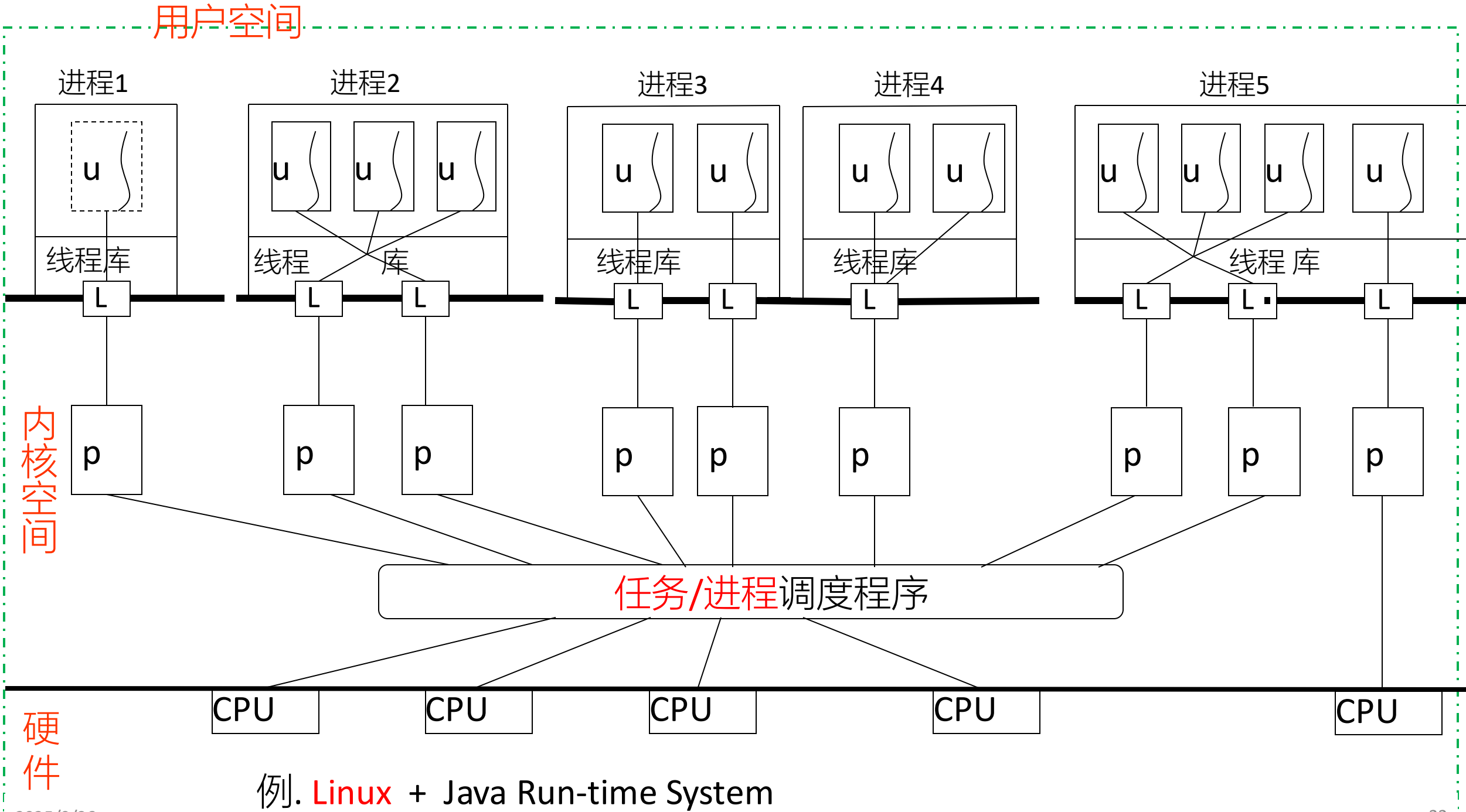
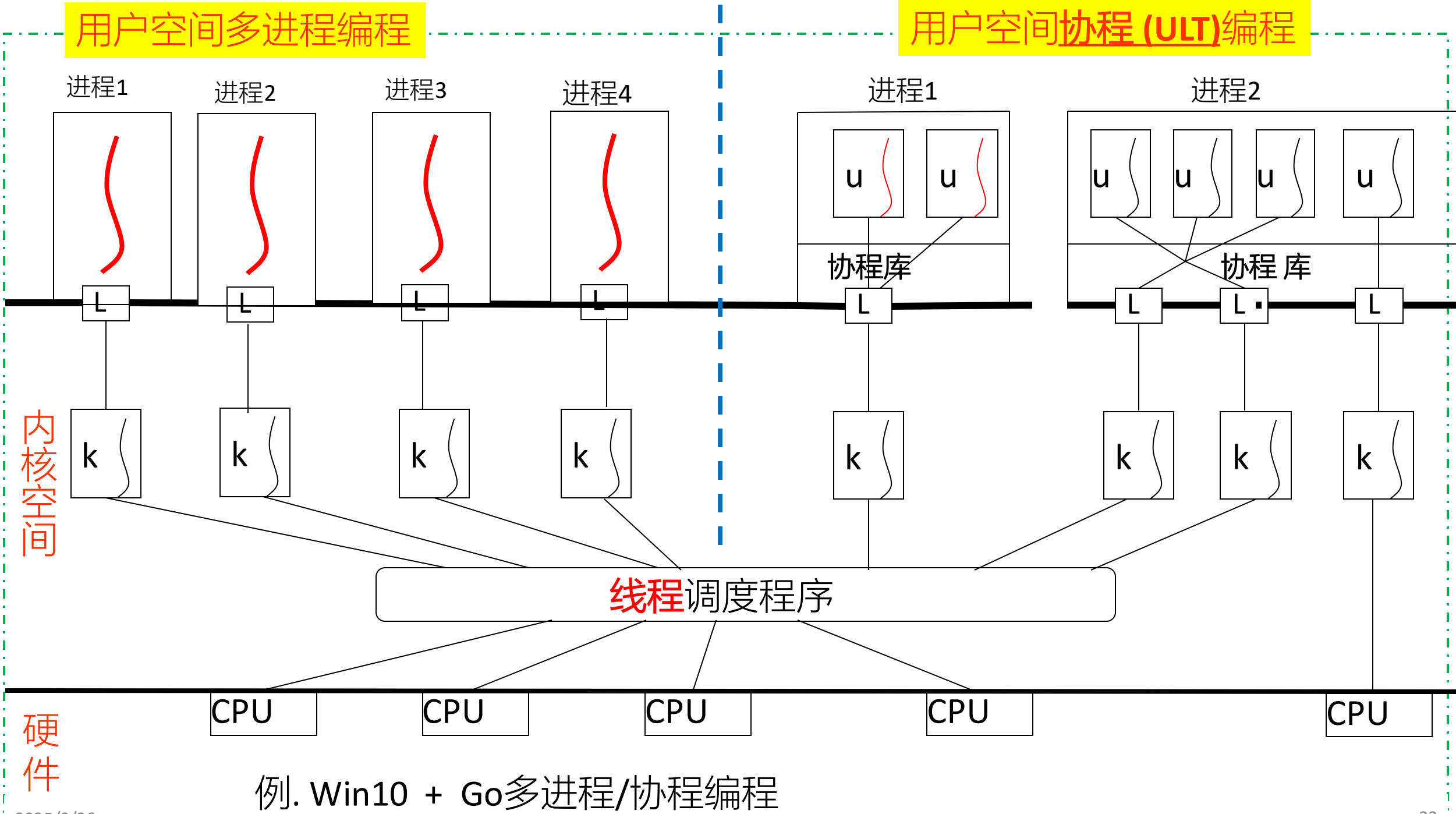


Fig. 4.0.3 例. Window 10 + Java Run-time System


用户空间
进程1
进程2
进程3
进程4
进程5
u
线程库
线程 库
L
内核空间
p
任务/进程调度程序
硬件
CPU
例. Linux + Java Run-time System



例. Win10 + Go多进程/协程编程

4.4 Thread Libraries

- A thread library provides the programmer an API for creating and managing thread
- Two approaches for thread library implementation

ULT-based	KLT-based
▫ provide a library entirely in user space with no kernel support ▫ codes and data structures for the threads exist in user space ▫ invoking a function in the library results in a local function call in user space ▫ e.g. thread in C++, Java	▫ implement a kernel-level library support directly by OS ▫ codes and data structures for the threads exist in kernel space ▫ invoking a function in the API for the library results in a system call to the kernel ▫ e.g. thread in Windows

Linux Thread

- Linux refers to them as tasks rather than threads
- Thread creation is done through clone() system call
  - clone() allows a child task to share the address space of the parent task (process)

- 严格意义上, Linux内核只支持教科书所述进程, 并不支持线程. 因为Linux中只有进程的PCB所对应的task_struc, 没有线程的TCB

- For more details , refer to Linux as Case Study-4-Threads in Linux

- 在 Linux 系统中，线程和进程的区别是：Linux 的线程机制和 Windows 等其他操作系统的很不一样。Linux 中没有为线程设置专门的数据结构，也没有专门的线程调度算法。在 Linux 内核看来，线程就是一个进程，只是一个和其他进程共享资源的特殊进程而已

POSIX/Linux Thread Library

- LinuxThreads 项目
  - Linux 最初开发时，在内核中并不能真正支持线程。但可以通过 clone() 系统调用将进程作为可调度的实体
  - clone()调用创建了调用进程(calling process)的一个拷贝，这个拷贝与调用进程共享相同的地址空间
  - LinuxThreads 项目使用clone()调用完全在用户空间模拟对线程的支持
  - 采用one-to-one mapping模型

- Pthreads/POSIX
  - a specification for thread behavior, on the basis of POSIX standard (IEEE 1003.1C)
  - for programmer, it defines an API for thread creation and synchronization
  - may be provided as either a user- or kernel-level library
  - common in UNIX operating systems (Solaris, Linux, Mac OS X), not by Windows

POSIX/Linux Thread Library

- Phtread/POSIX线程库定义了线程属性对象，封装了线程的创建者可以访问和修改的线程属性，主要包括
  - 作用域（scope）
  - 栈尺寸（stack size）
  - 栈地址（stack address）
  - 优先级（priority）
  - 分离状态（detached state）
  - 调度策略和参数（scheduling policy and parameters）

- E.g. 线程创建

```
int pthread_create(pthread_t *tid,
                   /*线程ID
                   const pthread_attr_t *attr,
                   /*线程属性
                   void *(*start_rtn)(void),
                   /* 线程执行路径、轨迹
                   void *arg
                   /*传递给线程的参数
                   )
```

创建成功时，返回0

- Example

1. 线程创建 pthread_create()

调用格式：

```
#include<pthread.h>
int pthread_create(pthread_t *thread，pthread_attr_t*attr，void*(*start_routine)(void *), void
                   *arg)
```

参数：

thread：指向所创建线程的标识符的指针，线程创建成功时，返回被创建线程的 ID

attr：用于指定被创建线程的属性，NULL 表示使用默认属性

start_routine: 函数指针，指向线程创建后要调用的函数，是一个以指向 void 的指针作 为参数和返回值的函数指针，这个被线程调用的函数也被称为线程函数

arg：指向传递给线程函数的参数

返回值：

创建成功：0

创建失败：返回错误码

```c
/* thread_create.c */
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>

/*线程函数1*/
void *mythread1(void)
{
    int i;
    for(i=0;i<5;i++)
    {
        printf("I am the 1st pthread,created by mybeilef321\n");
        sleep(2);
    }
}
/*线程函数2*/
void *mythread2(void)
{
    int i;
    for(i=0;i<5;i++)
    {
        printf("I am the 2st pthread,created by mybelief321\n");
        sleep(2);
    }
}

int main()
{
    pthread_t id1,id2;  /*线程ID*/
    int res;
    /*创建一个线程，并使得该线程执行mythread1函数*/
    res=pthread_create(&id1,NULL,(void *)mythread1,NULL);
    if(res)
    {
        printf("Create pthread error!\n");
        return 1;
    }
    /*创建一个线程，并使得该线程执行mythread2函数*/
    res=pthread_create(&id2,NULL,(void *)mythread2,NULL);
    if(res)
    {
        printf("Create pthread error!\n");
        return 1;
    }
    /*等待两个线程均推出后，main()函数再退出*/
    pthread_join(id1,NULL);
    pthread_join(id2,NULL);

    return 1;
}
```

```
song@ubuntu:~/lianxi$ vim thread_create.c
song@ubuntu:~/lianxi$ gcc thread_create.c -o thread_create -lpthread
song@ubuntu:~/lianxi$ ./thread_create
I am the 2st pthread,created by mybelief321
I am the 1st pthread,created by mybeilef321
I am the 2st pthread,created by mybelief321
I am the 1st pthread,created by mybeilef321
I am the 2st pthread,created by mybelief321
I am the 1st pthread,created by mybeilef321
I am the 2st pthread,created by mybelief321
I am the 1st pthread,created by mybeilef321
I am the 2st pthread,created by mybelief321
I am the 1st pthread,created by mybeilef321
```

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[]) {
   int i, scope;
   pthread_t tid[NUM THREADS];
   pthread_attr_t attr;        /*线程属性集合*/
   pthread_attr_init(&attr); /* get the default attributes */
   /* first inquire on the current scope */
   if (pthread_attr_getscope(&attr, &scope) != 0)/*非零值，表示error
      fprintf(stderr, "Unable to get scheduling scope\n");
   else {
      if (scope == PTHREAD_SCOPE_PROCESS)
         printf(“PTHREAD_SCOPE_PROCESS”);    /*PCS */
      else if (scope == PTHREAD_SCOPE_SYSTEM)
         printf("PTHREAD_SCOPE_SYSTEM");     /*SCS */
                                              else
         fprintf(stderr, "Illegal scope value.\n");
```

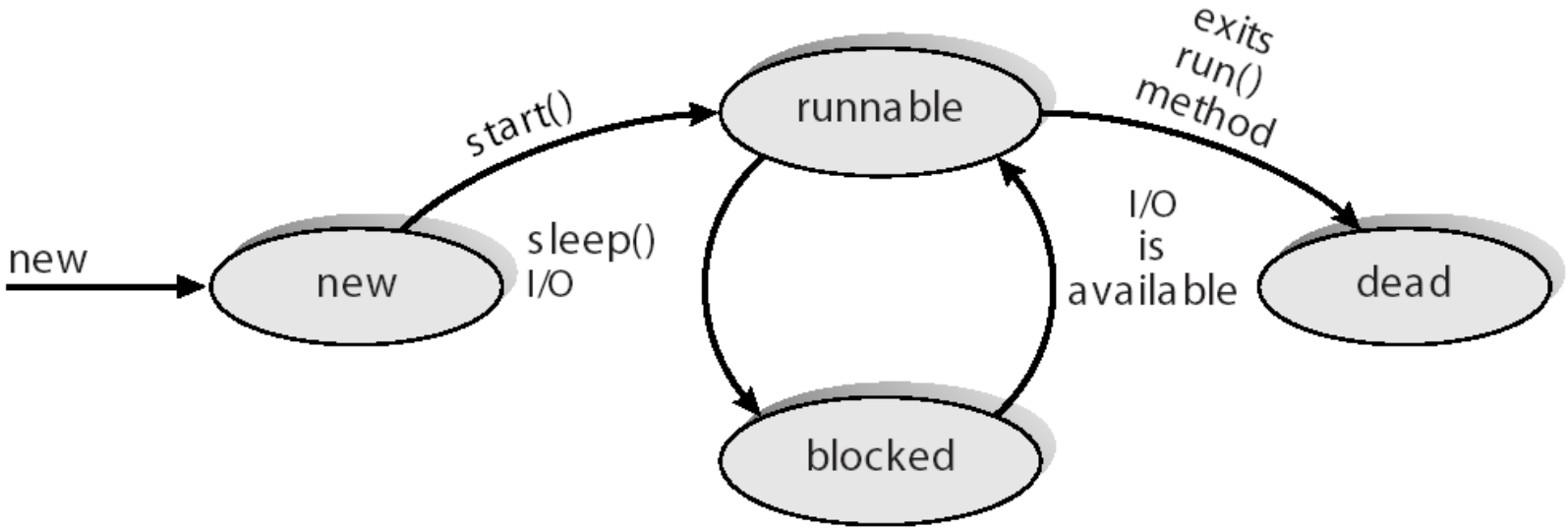
Windows Thread

- Win32 threads
  - kernel-level thread library available on Windows systems
  - implement the one-to-one mapping
- Each thread contains
  - a thread id
  - thread context
    - register set
    - separate user and kernel stacks
    - private data storage area
- The register set, stacks, and private storage area are known as the thread context
- The primary data structures of a thread include:
  - ETHREAD (executive thread block)
  - KTHREAD (kernel thread block)
  - TEB (thread environment block)

Java Thread Library

- Supporting thread creating and management in Java environments, managed by JVM

- Java thread API is typically implemented using a thread library available on the host system
  - e.g. on Windows systems, Java threads are implemented using the Win32 API

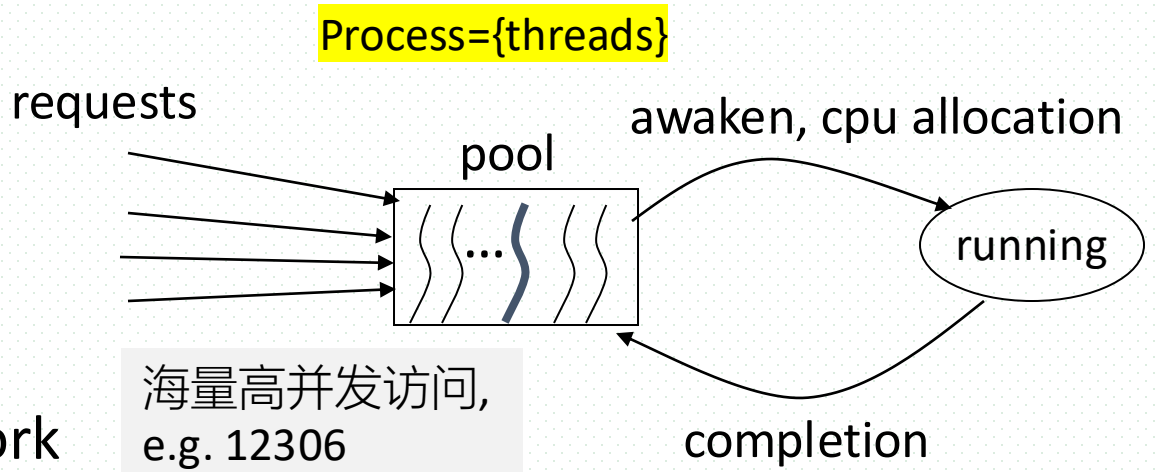


4.5 Implicit Threading[3.16]

- Thread pool/线程池
- ~~【openMP~~
- ~~Grand central dispatch】~~

Thread Pools

- Thread pool
  - a set of pre-created threads, the number of threads in this set is limited
- E.g. In a multithreading web server, create a number of threads at process startup and place them into a pool, where they sit and wait for work
  - when a server receives a request, it awakens a thread from this pool— if one is available— and passes it the request for service
  - once the thread completes its service, it returns to the pool and awaits more work
  - if the pool contains no available thread, the server waits until one becomes free
  - the number of threads in web server process is dynamic changing



海量高并发访问,
e.g. 12306

e.g. 500,000 ordering/s in 淘宝/天猫, 2021，双十一,
, Oceanbase分布式数据库

Thread Pools

- Thread pools offer these benefits
  - servicing a request with an existing thread is faster than waiting to create a thread.
  - a thread pool limits the number of threads that exist at any one point. This is particularly important on systems that cannot support a large number of concurrent threads-线程池中线程数目限定了可同时响应的外部请求
  - separating the task to be performed from the mechanics of creating the task allows us to use different strategies for running the task
    - for example, the task could be scheduled to execute after a time delay or to execute periodically
- Some operating systems and programming languages/environments provide the thread pool mechanisms
  - the Windows API provides several functions related to thread pools
  - thread pools in Java

```
DWORD WINAPI PoolFunction(AVOID Param) {
   /*
   * this function runs as a separate thread.
   */
}
```

How to determine the number of threads in pool

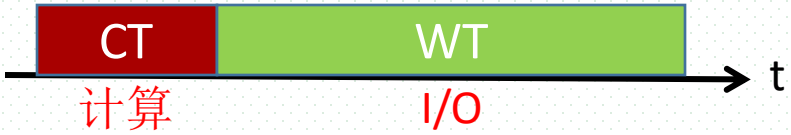
- The number of threads in the pool can be set heuristically based on factors such as
  - the number of CPUs in the system
  - the amount of physical memory
    - 线程栈对内存资源的占用 
  - the expected number of concurrent client requests

- More sophisticated thread-pool architectures can dynamically adjust the number of threads in the pool according to usage patterns
  - such architectures provide the further benefit of having a smaller pool— thereby consuming less memory— when the load on the system is low
  - /*自动感知业务负荷，动态调整线程池参数、规模、（内存）资源占用，提高系统资源利用率和计算速度
- We discuss one such architecture, Apple's Grand Central Dispatch, later in this section[略].

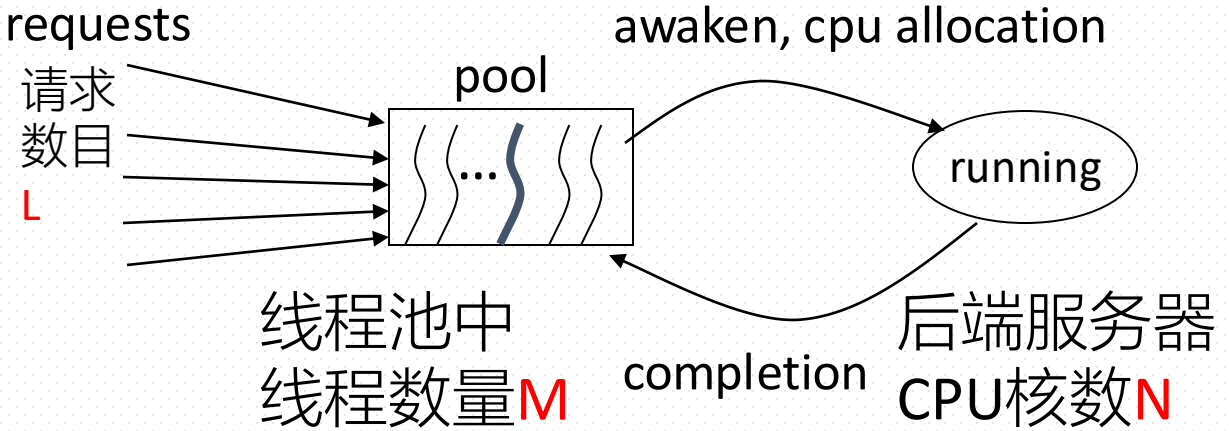
How to determine the number of threads in pool

- 根据服务器端的CPU核的数目N、线程需要执行的任务/进程类型，确定线程数目M
- 任务类型
  - CPU密集型，CPU-intensive，CPU-bound
    - e.g. 科学计算，矩阵计算

  - I/O密集型，I/O-intensive，I/O-bound
    - e.g. 数据库访问磁盘I/O，网络I/O



海量高并发访问



M设置方式

- CPU密集型任务：

  M=N

- I/O密集型任务：

  M = N*(1+WT/CT)

  = e.g. 4*(1+ 75/25)

CT: computing time
WT: waiting/IO time

CT: 计算任务执行时间
WT: I/O操作执行时间

openMP

- 基于共享内存模型的C/C++/Fortran编程环境中的并行编程机制
- OpenMP
  - a set of compiler directives as well as an API for programs written in C, C++ , or FORTRAN that provides support for parallel programming in shared-memory environments
    - process communication by shared memory
- OpenMP identifies in programs the parallel regions as blocks of code that may run in parallel, then application developers insert compiler directives into their code at parallel regions, and these directives instruct the OpenMP run-time library to execute the region in parallel

Grand Central Dispatch (GCD)/大中央调度

- A parallel programming environment/technology, used for Apple's Mac OS X and iOS operating systems
- A combination of extensions to the C language, an API, and a run-time library that allows application developers to identify sections of code to run in parallel
  - thread-pool architectures can dynamically adjust the number of threads in the pool according to usage patterns

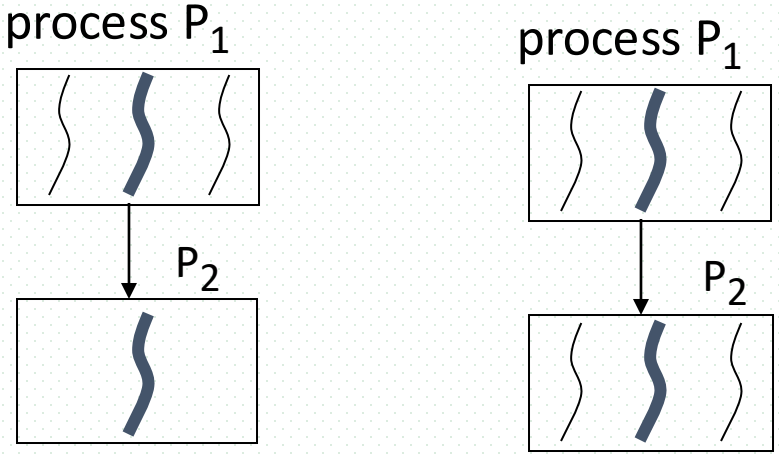
4.6 Thread Issues

in designing multithreading programs

- fork(), exec() for thread creation
- Thread cancellation
- **Signal handling**
- Thread local storage
- ~~Scheduler activations~~

fork(), exec()

- If one thread in the parent process P_1 calls fork to create a new child process P_2, the new process P_2 will
  - duplicate only the thread who create it, and becomes a single-thread process , or
  - duplicate all threads in the parent process



- System calls for task/process/thread creation in Linux
  - fork(), vfork()
  - clone() + exec()

Thread Cancellation

- Thread cancellation
  - involves terminating a thread before it has completed
  - e.g. for example
    - if multiple threads are concurrently searching through a database and one thread returns the result, the remaining threads might be canceled
- Target thread
  - the thread to be cancelled

```
pthread_create(&tid, 0, worker, NULL)
pthread_cancel(tid)
```

- Cancellation of target thread in two scenarios
  - asynchronous cancellation
    - one thread immediately terminates the target thread
  - deferred cancellation
    - the target thread periodically checks whether it should terminate, allowing it an opportunity to terminate itself in an orderly fashion, such as waiting for the terminating of its child threads

Signal (信号) Handling

- Definition in Unix
  - a facility provided by OS to notify a process that a particular event occurred
  - event-driven mechanism
- Handling of signals in OS
  - a signal is generated by the occurrence of a particular event, and received by OS, e.g. Ctrl + Alt + Delete
  - a generated signal is delivered to a process
  - once delivered, the signals must be handled
- Signals vs interrupts
  - the interrupt is processed by CPU
  - the signal is processed by OS

Signal Handling

- Two types of signal
  - synchronous signals, generated by an internal event in the running/receiver process
    - e.g. illegal memory access, division by zero, trap??
  - asynchronous signals, generated by an event external to a running/receiver process
    - e.g. terminating a process with signal *kill*, or having a timer expire
- Example: Linux下，用kill终止/关闭进程
  - 原理：1. 向操作系统内核发送一个终止信号和被终止进程的PID； 2.内核根据发送的终止信号类型，对进程进行相应的终止操作
  - 语法： kill [信号类型] 进程PID
    - kill -9 进程PID：强制结束进程，忽略进程间依赖关系
    - kill -2 进程PID：结束进程，但是并不是强制性的，常用的组合键发出的就是kill -2信号
    - kill -15进程PID：正常结束进程，是kill的默认选项。父进程在终止自身时，同时关闭子进程，释放内存

Signal Handling

- Each signal can be handled by
  - a default signal handler that is run by the kernel, or
  - a user-defined signal handle

- Windows2000 has a IPC mechanism, called asynchronous procedure calls (APCs), similar to asynchronous signal in Unix

Thread-Local Storage (线程本地存储)

- Threads belonging to a process share the data of the process, and this data sharing provides one of the benefits of multithreaded programming
  - 基于共享数据的线程间通信
- However, in some circumstances, each thread might need its own copy of certain data. We will call such data thread-local storage (or TLS)
- For example, in a transaction-processing system, we might service each transaction in a separate thread. Furthermore, each transaction might be assigned a unique identifier. To associate each thread with its unique identifier, we could use thread-local storage

- TLS data are unique to each thread. Most thread libraries provide some form of support for thread-local storage, including
  - Windows, Pthreads, Java

Scheduler Activations/激活

- Communication between the kernel and the thread library, which may be required by the many-to-many and two-level models discussed in Section 4.3.3
- Such coordination allows the number of kernel threads to be dynamically adjusted to help ensure the best performance

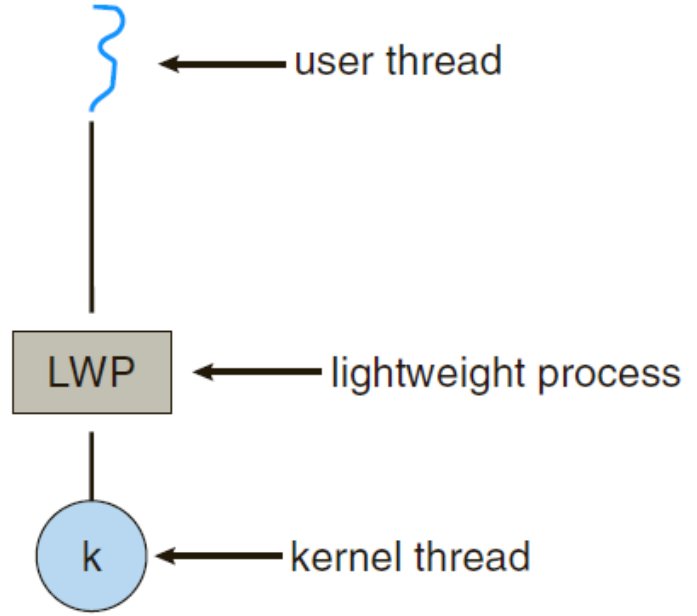


Figure 4.13 Lightweight process (LWP).

4.7 OS Examples

- As shown in  , data in threads includes
  - shared data
  - thread-specific data

- E.g. POSIX线程属性主要包括
  - 1. 作用域（scope）
  - 2. 栈尺寸（stack size）
  - 3. 栈地址（stack address）
  - 4. 优先级（priority）
  - 5. 分离的状态（detached state）
  - 6. 调度策略和参数（scheduling policy and parameters）

Windows Thread

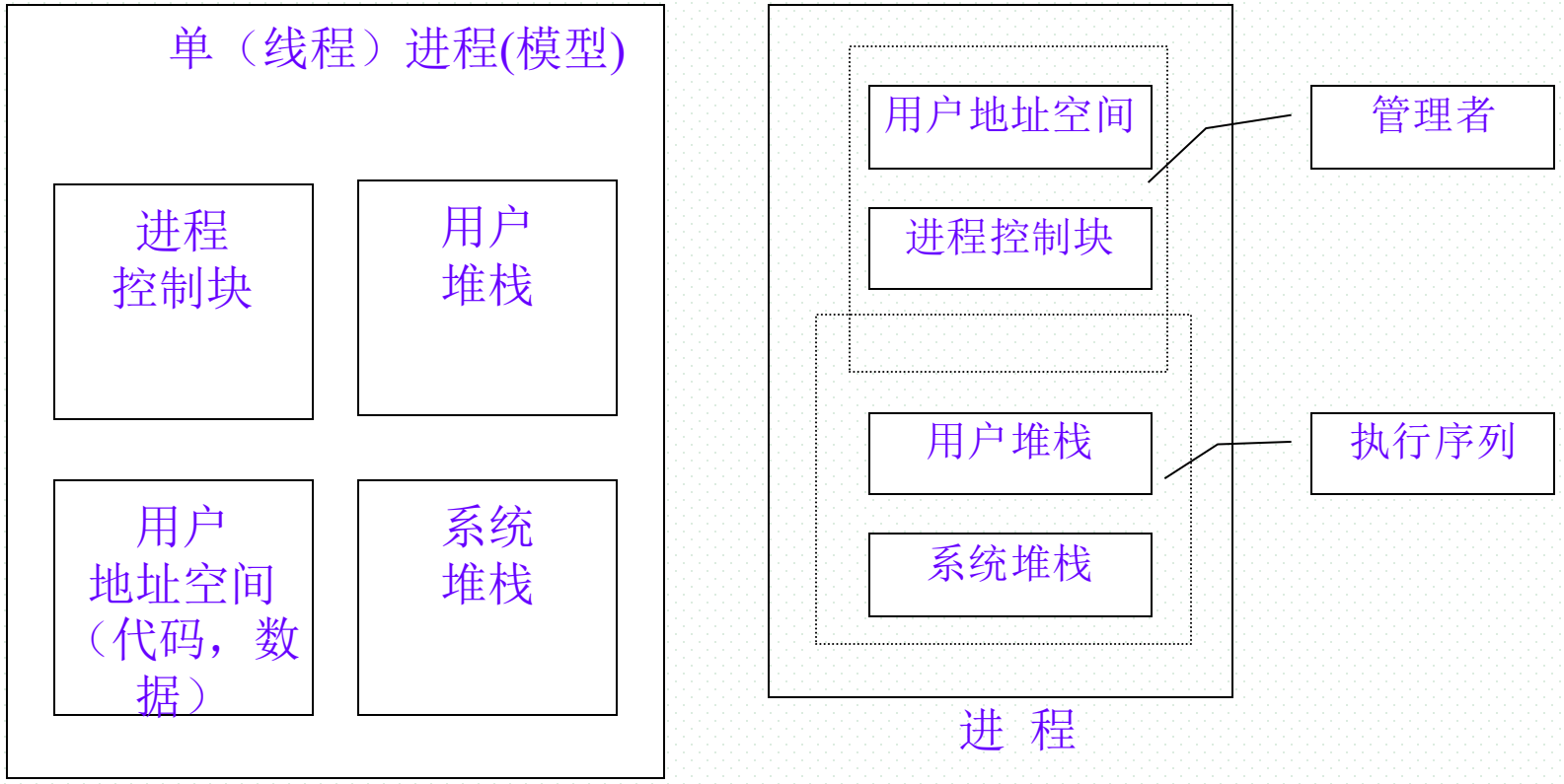
- Win32 threads
  - kernel-level thread library available on Windows systems
  - implement the one-to-one mapping
- Each thread contains
  - a thread id
  - register set
    - contents of register set for thread switch，representing the CPU states, including that of PC, integer and float-point registers, and architecture-related registers
  - separate user and kernel stacks, for thread executing in kernel mode and user mode respectively
  - private data storage area
- Thread context
  - register set
  - Stacks
  - private storage area (heap, malloc)
- The primary data structures of a thread include:
  - ETHREAD (executive thread block)
  - KTHREAD (kernel thread block)
  - TEB (thread environment block)
- Thread states
  - ready, standby, running, waiting, transition, terminated

Appendix 4-1 关于线程

1. 为什么引入线程

- 操作系统中引入进程是为了支持多道程序的并发执行，提高程序运行效率和系统资源利用率
- 传统基于进程的操作系统（又称作单线程结构进程）给并发程序设计效率带来问题
  - 进程切换时进程管理的时空开销过大，频繁的进程调度消耗CPU时间，为每个进程分配存储空间限制了操作系统中进程的总数
  - 进程通信代价大
  - 进程之间的并发性粒度较粗，并发度不高，不适合并行计算和分布并行计算的要求
  - 不适合客户/服务器计算的要求

单（线程）进程的内存布局和运行

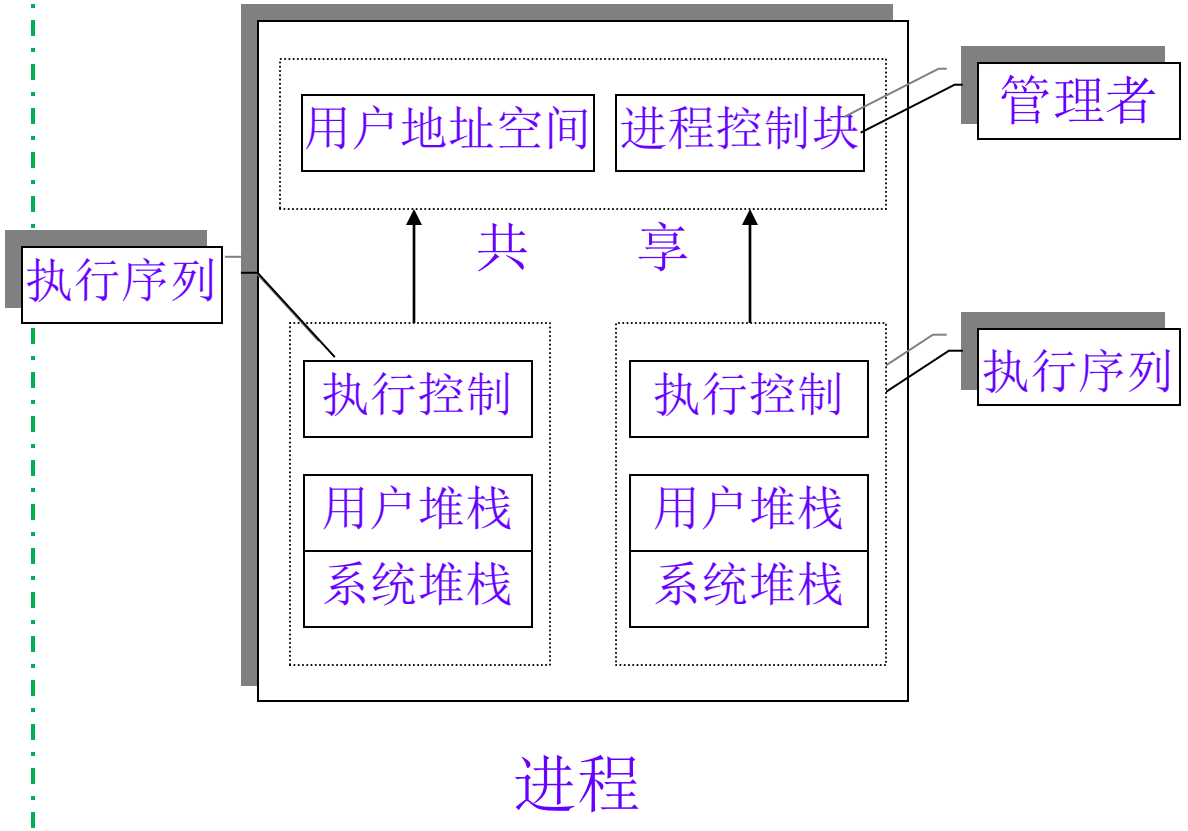


关于线程

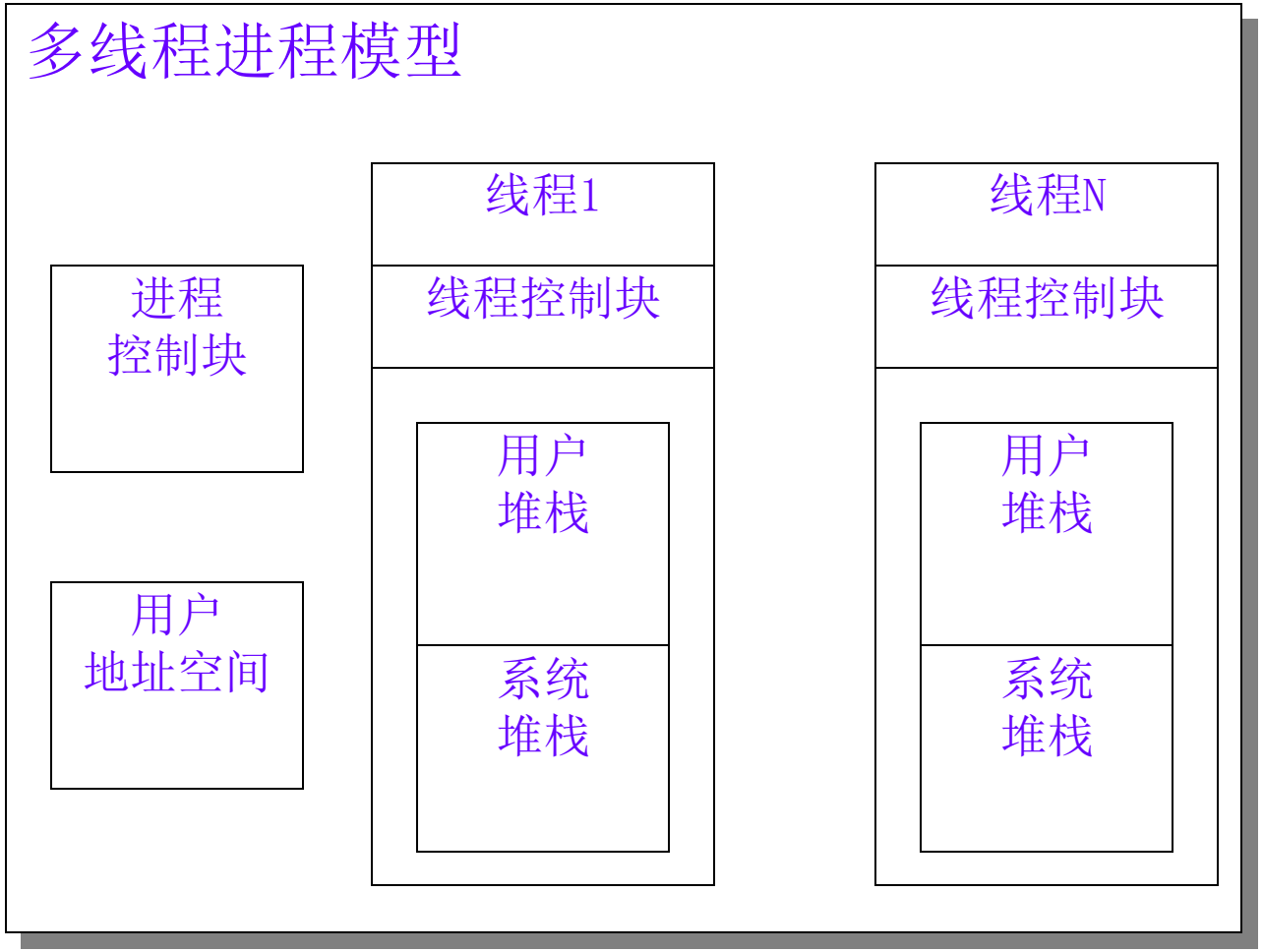
- **解决问题的思路**：

  将进程的两项功能－－“独立分配资源”与“被调度分派执行”分离开

  - 进程作为系统资源分配和保护的独立单位，不需要频繁地切换
  - 一个进程可以包含多个线程
  - 线程作为系统调度和分派的基本单位，能轻装运行，会被频繁地调度和切换，在这种指导思想下，产生了线程的概念



管理和执行相分离的进程模型



多线程进程的内存布局：

前提：1个进程内多个线程执行类似、相关工作，有关联性，可以共享内存、数据等资源

关于线程

- **2. 基本概念**
- Thread定义
  - 进程内一个相对独立的、可执行单元，是进行处理器调度和分派的基本单位
  - 使用进程分配给它的资源和环境
- 线程的性质
  - 进程内相对独立的可执行单元，类似于应用中子任务的执行
  - OS基本调度单元，包含调度所需信息
  - 进程的执行体现为进程内线程的执行，创建进程时创建进程内的线程，线程可创建其它线程
  - 进程拥有资源，同一进程内的线程共享并使用资源
  - 同一进程内的线程间的切换成本小
  - 共享资源的线程间的通信和同步机制
  - 线程的生命周期和状态变化

多线程环境中进程、线程的定义

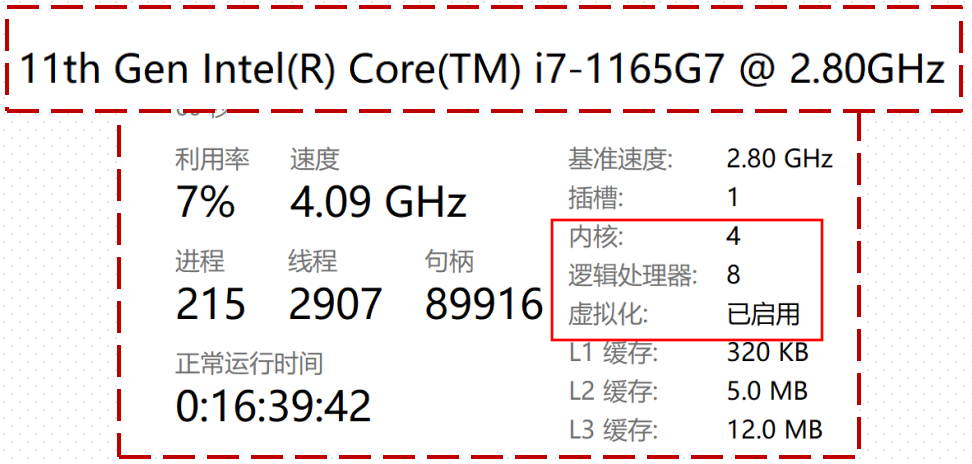
- **进程**是操作系统中进行保护和资源分配的基本单位，具有
  - 一个虚拟地址空间，用来容纳进程的映像
  - 对处理器、其它(通信的)进程、文件和I/O资源等的存取保护机制
- **线程**是操作系统进程中能够独立执行的实体（控制流），是处理器调度和分派的基本单位
  - 线程是进程的组成部分，每个进程内允许包含多个并发执行的实体（控制流）

- **线程优点**
  - 一个进程内的多个线程驻留同一地址空间，共享数据和文件
  - 线程创建/撤销的代价比进程创建/撤销的代价小——不需要频繁分配、释放资源，提高了并发程序执行效率
  - CPU在同一个进程内的各个线程间的切换代价比在进程间的切换的代价小
  - 线程间通信的有效性
  - 方便简化用户的程序结构工作
  - 支持多线程机制的语言和开发环境——Java、C#

Appendix 4-1 Hyper Threading (HT, 超线程)技术

1. Concept/Principles

- Hyper Threading
  - a CPU speedup method first appearing in P4 3.08G (in 2002.11. ?)
  - virtualizing （逻辑虚拟化） a physical CPU as two logical CPUs, i.e. two logical CPUs multiplex physical resources of one physical CPU.
    - 在一个物理核上模拟两个逻辑核
    - 两个逻辑核具有各自独立的寄存器（eax、ebx、ecx、msr等等）和APIC，但共享使用物理核的执行资源，包括执行引擎、L1/L2缓存、TLB和系统总线等等
  - from the viewpoint of OS or programmers, two threads can run in parallel on these two logical CPUs



HT 对性能的影响分析

- HT在一个物理核上使用两个物理任务描述符，单个物理核的计算能力并没有增加
- HT通过两个逻辑核对一个物理核上的计算资源高效复用，提高了进程/线程的整体吞吐量
- E.g. 采用多线程编程的Web application，在超线程HT支持下，两个被调度到同一物理核的不同超线程HT上的应用线程，通过共享cache, TLB，大幅降低任务切换的开销——无需OS参与
- 另外，在某个线程不忙的时候，超线程允许其线程使用物理计算资源，有助于提升物理核整体吞吐量
  - 但由于超线程对物理核执行资源的争抢，业务的执行时延也会相应增加

性能影响定量评测

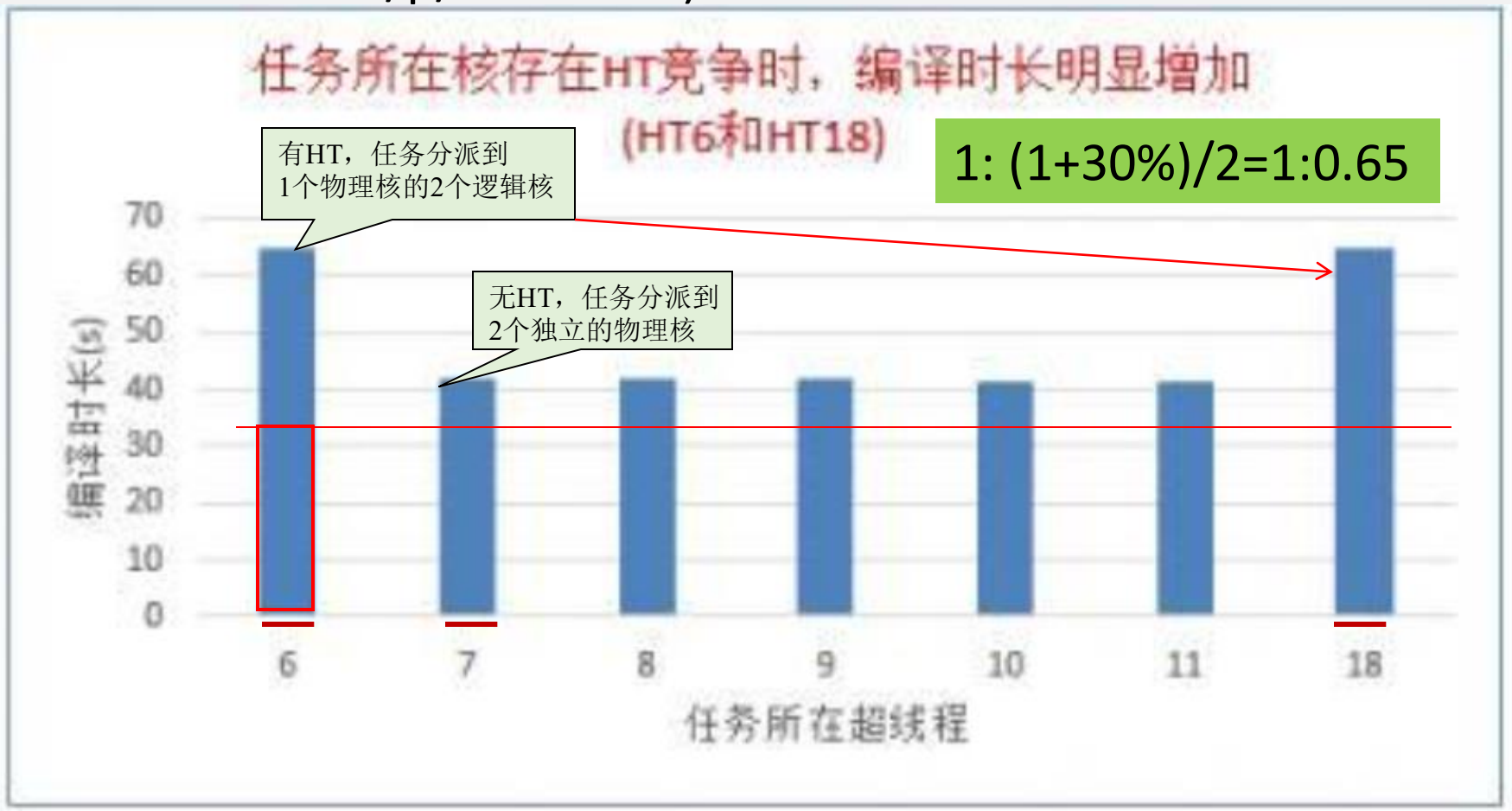
- 从Intel和VMware对外宣称资料以及一些实际评测
  - 开启超线程后，物理核总计算能力是否提升以及提升的幅度与业务模型密切相关，平均**提升在20%-30%**左右

- 当同一CPU核上的两个超线程HT相互竞争CPU核的计算资源时，例如将2个对立无关、可完全并行执行的进程/程序安排到同一个物理核的2个HT上，启用超线程的计算能力相比不开超线程、并将两个进程安排到不同的CPU物理核的性能/速度会**下降约30%**
  - 将有HT竞争时的计算时间折半作为逻辑核的计算时间，与无HT竞争时的物理核计算时间进行比较

1: (1+30%)/2=1:0.65

示例——C++编译任务在单核无HT vs 单核有HT

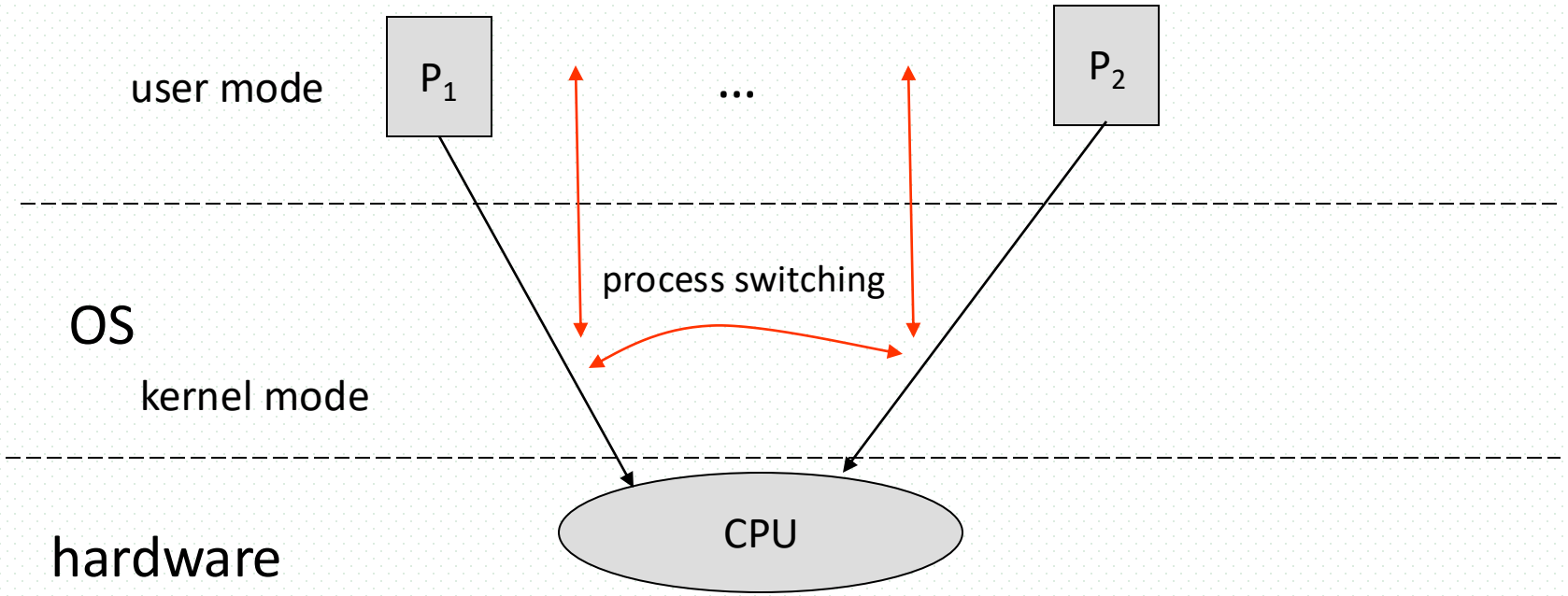
(HT6, HT18：2个任务安排到同一物理核的2个超线程HT/逻辑核；https://zhuanlan.zhihu.com/p/33324549)



HT6/2=HT18/2=66/2=33，相当于单个逻辑核的速度；
HT7=40，相当于单个物理核的速度

HT Principles

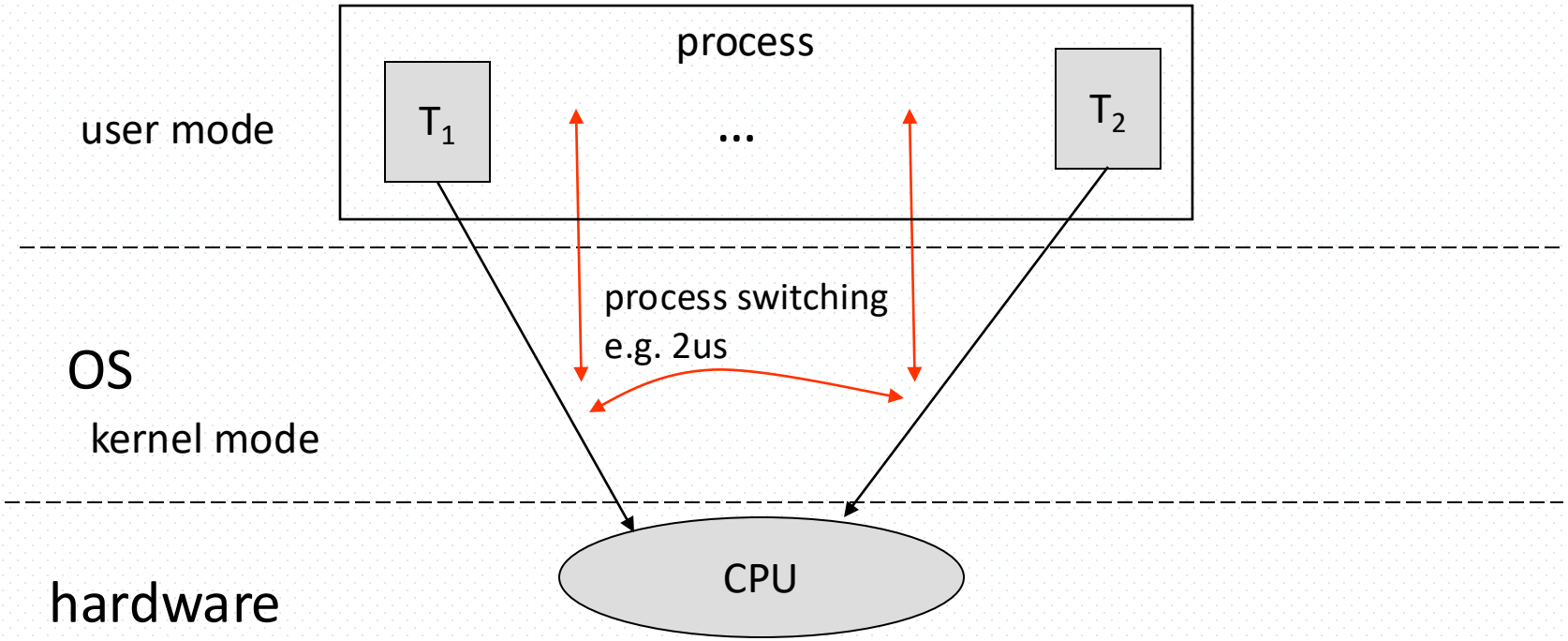
- Concurrent executing on process-level
- From the viewpoint of OS
  - PCB, process scheduling on a single CPU,
  - process (context) switching, higher



HT Principles

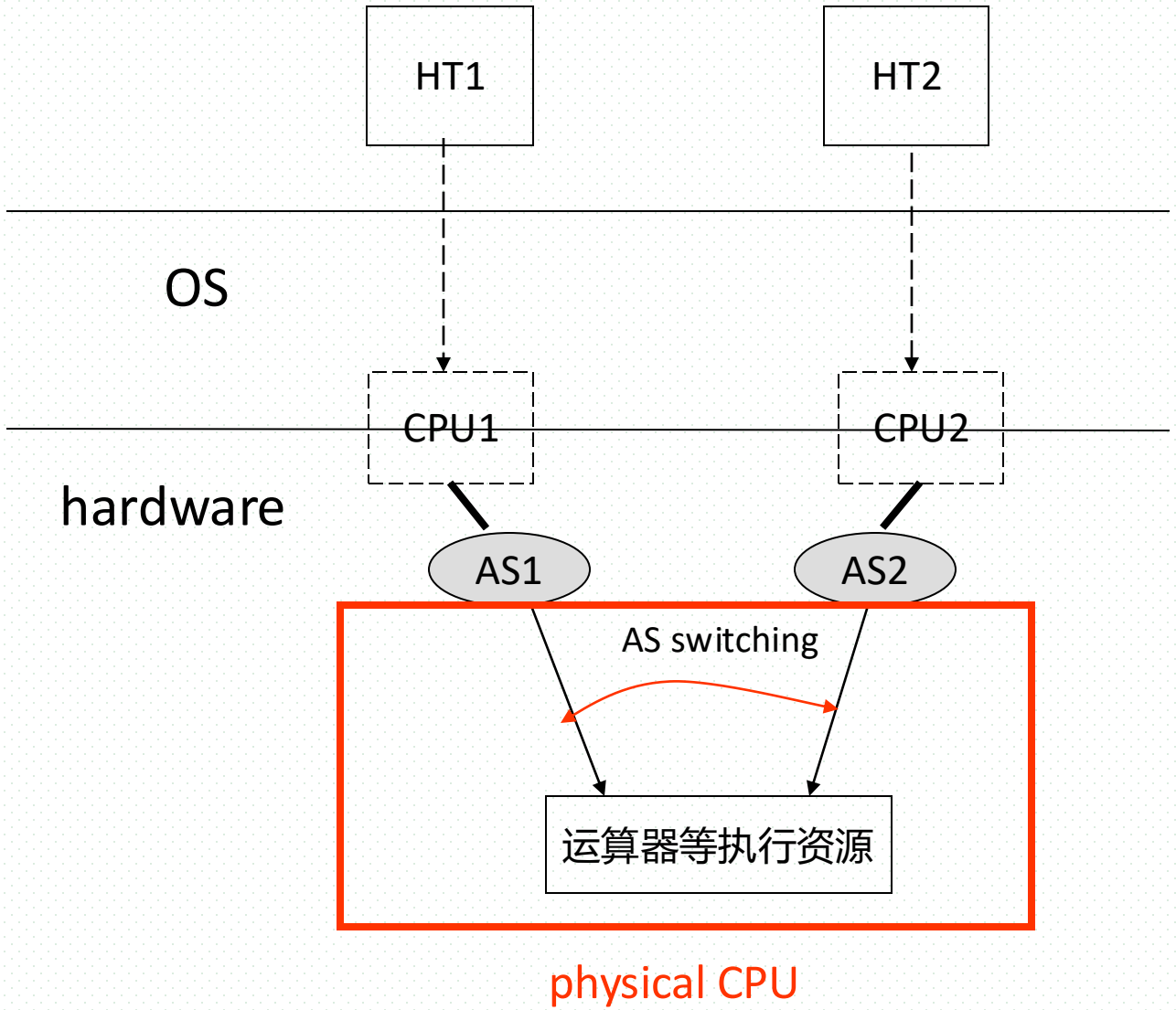
- Concurrent executing on thread-level

- From the viewpoint of OS
  - TCB, thread scheduling on a single CPU,
  - thread (context) switching, lower costs of process switching

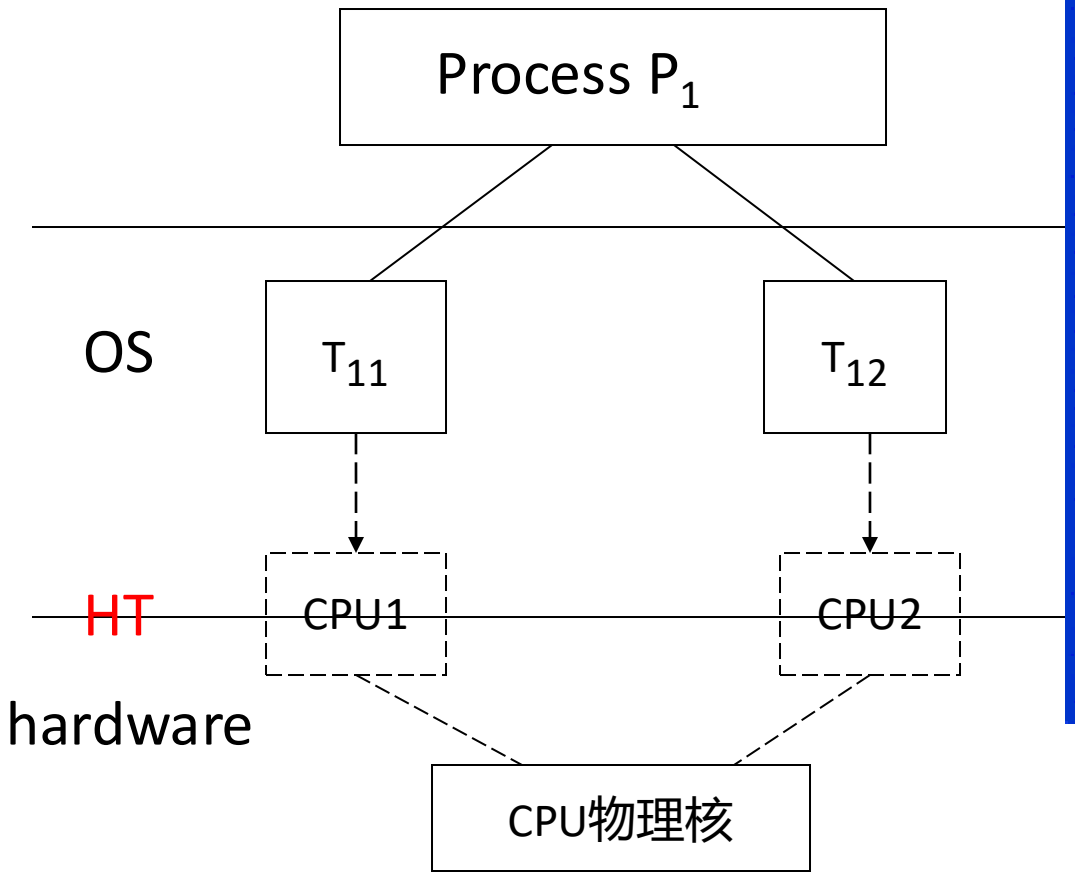


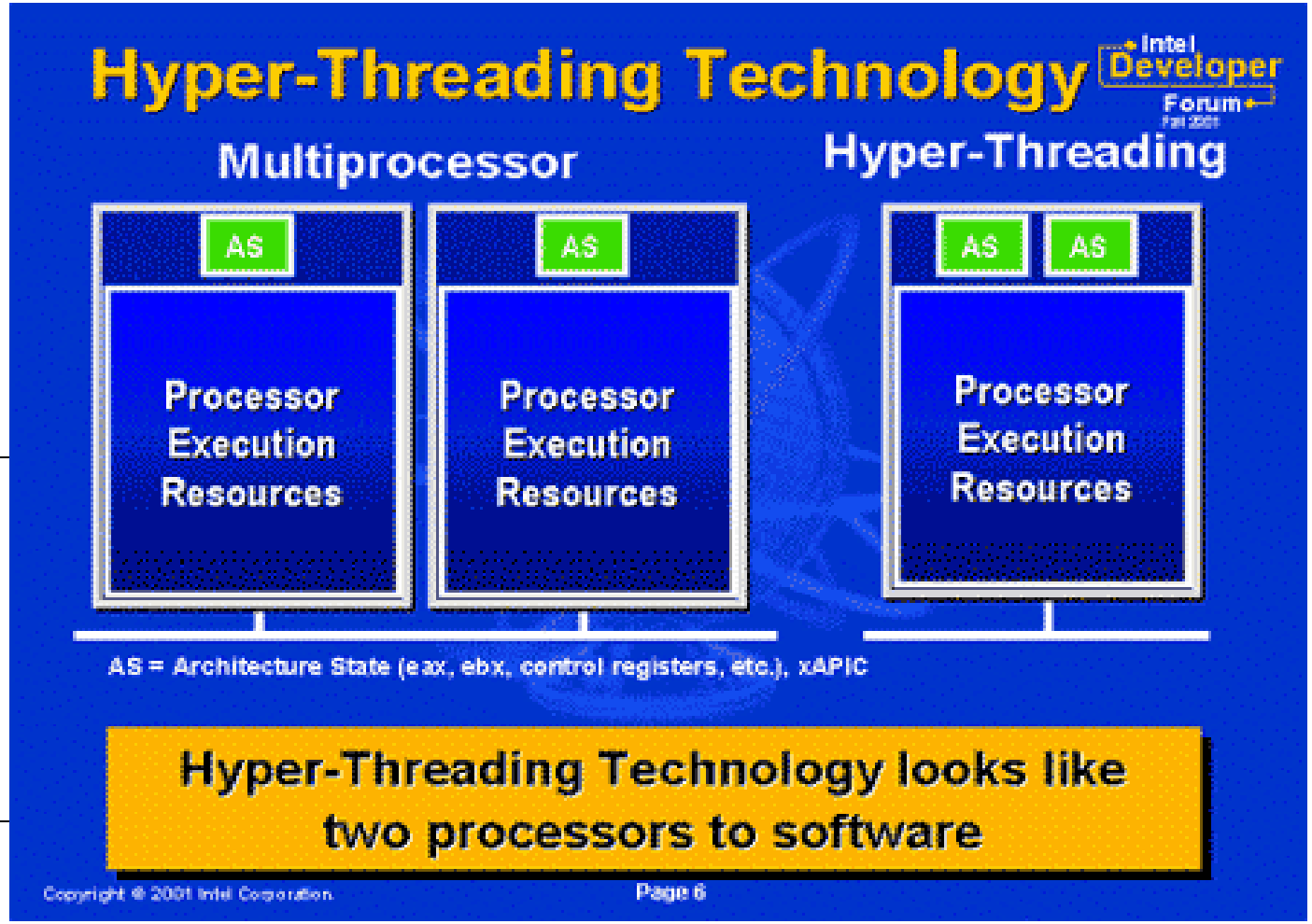
Principles：HT级的（虚拟）并行执行，无OS级的context switch

- 在1个物理芯片上模拟出2个逻辑CPU内核，每个逻辑CPU内核由AS (architecture state) 代表
  - 1个物理芯片两套AS寄存器
- AS，architecture state
  - 包括数据寄存器、段寄存器、控制寄存器、排错寄存器、信号特定寄存器(MSR)
- 对OS和线程而言可作为1个独立的CPU芯片来使用



Virtual paralleling (虚拟并行)





Principles

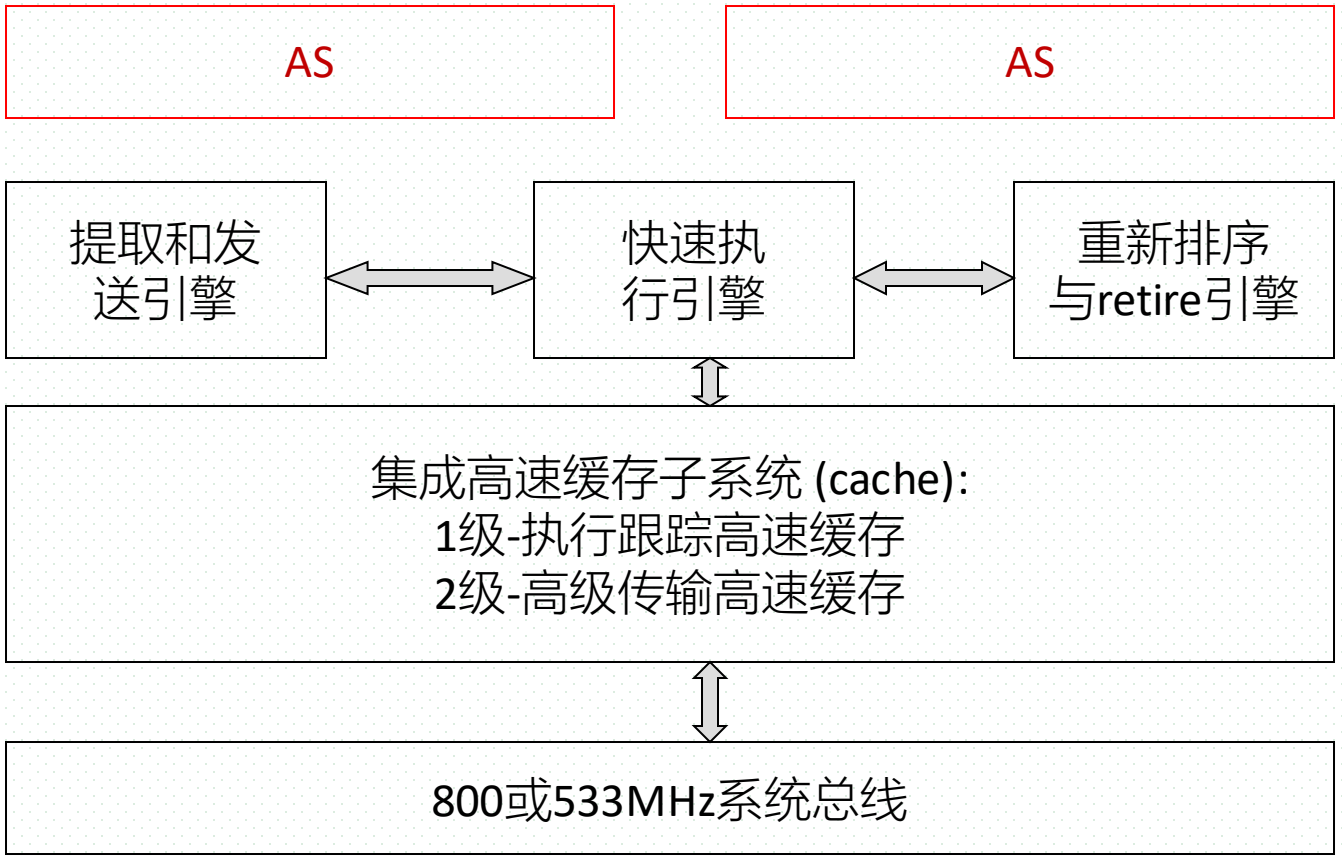
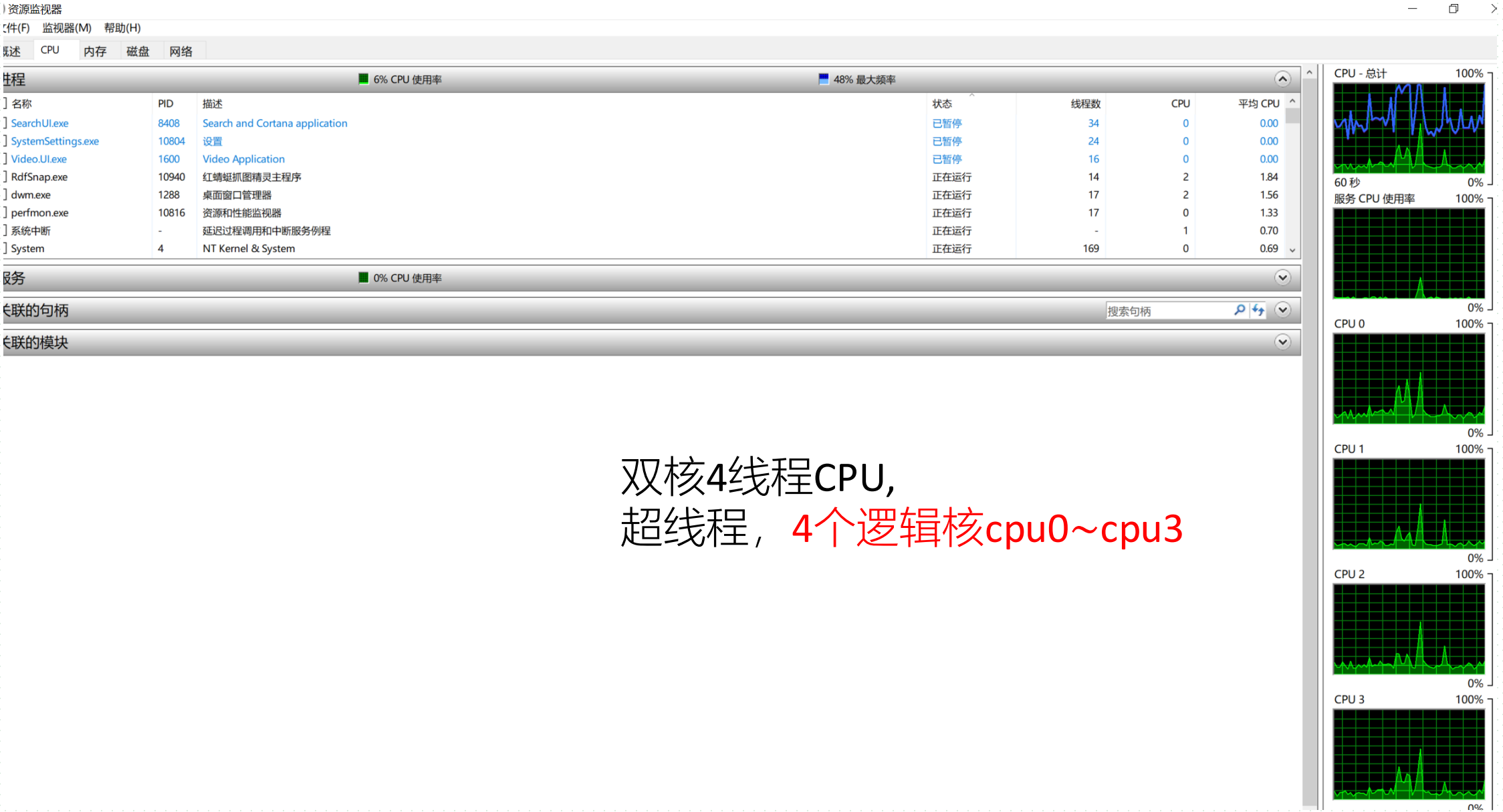


图 含超线程技术的Intel 处理器

Principles

- 从OS角度，线程T_1和T_2在2个逻辑CPU上并行执行，不存在switching
- 从CPU角度，利用AS（Architecture State)复制出2个逻辑CPU，AS反映了各逻辑CPU上所执行的各HT的状态，或各HT的context
- 两个逻辑处理器各自具有独立的微程序指针，但共享同一物理处理器的资源（如运算器，微码存储器，高速缓存，总线）
  - 每个逻辑处理器都可独立响应中断
- 物理CPU 上switching
  - 2个AS间的switching，或2个逻辑CPU间的switching，由CPU内部硬件机制完成
- AS switching的产生：一个物理处理器在指令（或机器周期）一级交替执行多个HT内的各指令

Example:　Windows 任务管理器

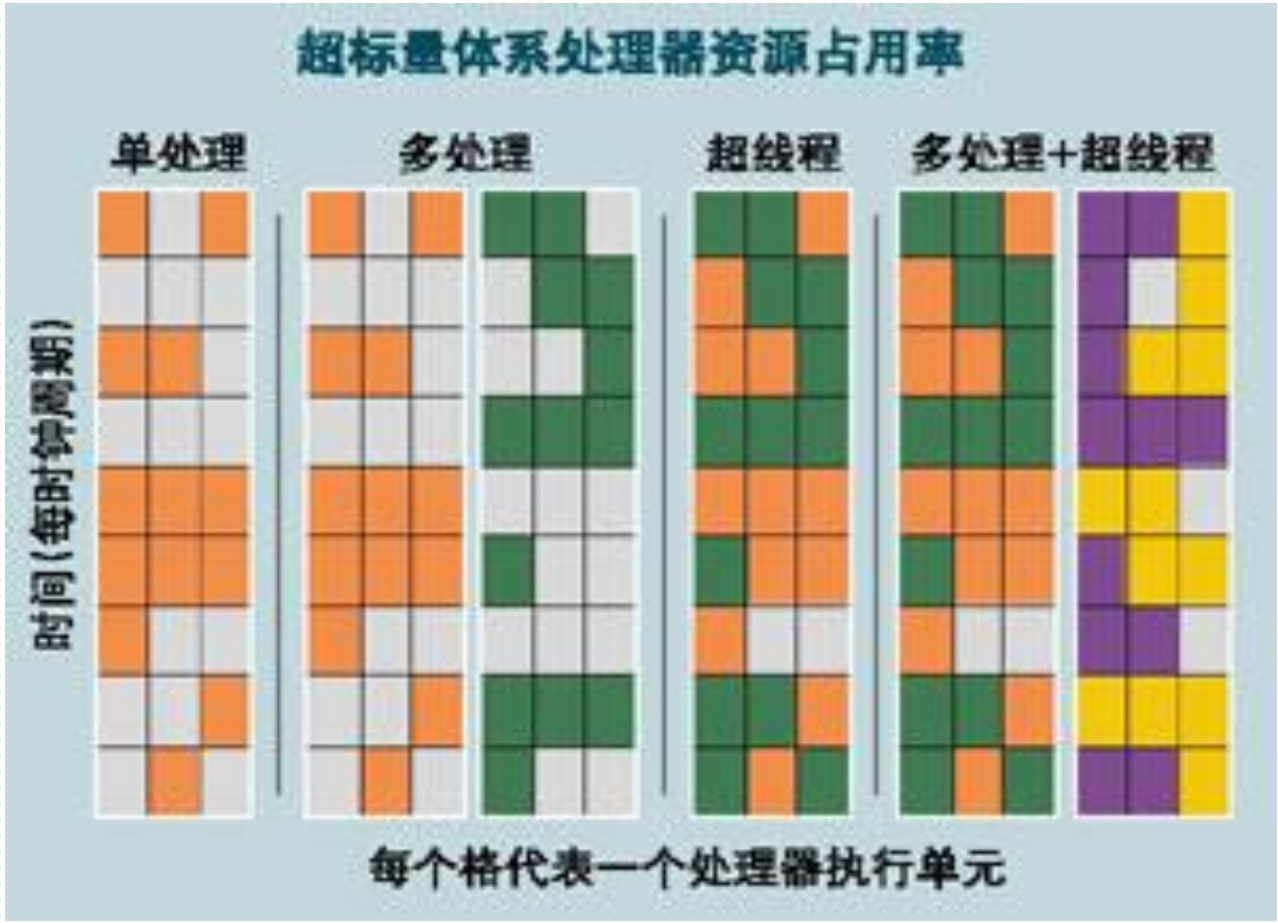


双核4线程CPU,
超线程，4个逻辑核cpu0~cpu3

3. Implementation

- 提高CPU处理速度的方法
  - 提高主频，使CPU在每一时钟周期内完成更多工作
- HT Method
  - reduce spare periods in CPU execution, so CPU can do more work in a clock period
  - 在CPU硬件内部，指令（或机器周期）一级的并发执行，提高CPU内部各级处理单元的利用率
- Supported by the CPU architecture, OS , compilers , and parallel programming for application programs
- 线程级的并行，指令级的并发

- 超线程HT：
  - 多条指令流水线并行执行，分别同时占用CPU物理核的不同执行单元



指令执行步骤：取指令-译码-取操作数-计算-存结果
每个步骤耗费1个或多个指令周期，占有CPU内部不同计算资源/处理器执行单元（如运算器，微码存储器，高速缓存，总线接口，寄存器）

4. 问题： 何时应该开启/关闭超线程，攒机时是否需有必要购买带HT的CPU

- 超线程应该关闭还是开启，主要取决于应用任务
  - 对于时延敏感性任务，比如用户需要及时等待任务运行结果的，在节点负载过高，引发超线程竞争时，任务的执行时长会显著增加，影响用户体验，建议关闭HT，保证用户任务一旦被OS调度得到CPU，可以得到独立的物理核
  - 对于后台计算型任务，比如超算中心上运行的后台计算型任务（一般要运行数小时或数天），建议开启HT，以提高整个节点的吞吐量

Appendix 4-2 多核Multicore CPU

1. 主要微处理器CPU

- Intel， AMD
- 嵌入式通用CPU
  - ARM, 高通, 苹果A6/7/8, 联发科, 华为海思麒麟, 龙芯, …
  - Intel 移动、低功耗处理器
    - 低压移动处理器，标压桌面处理器
  - 专用芯片：显卡nVIDIA, AMD; Texas instrument

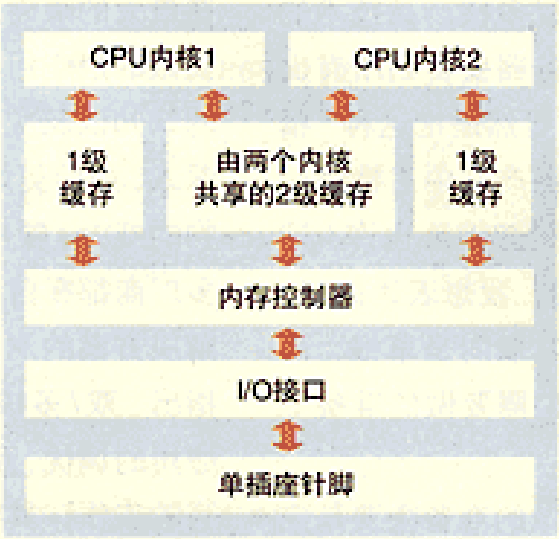
2. 双核/多核CPU

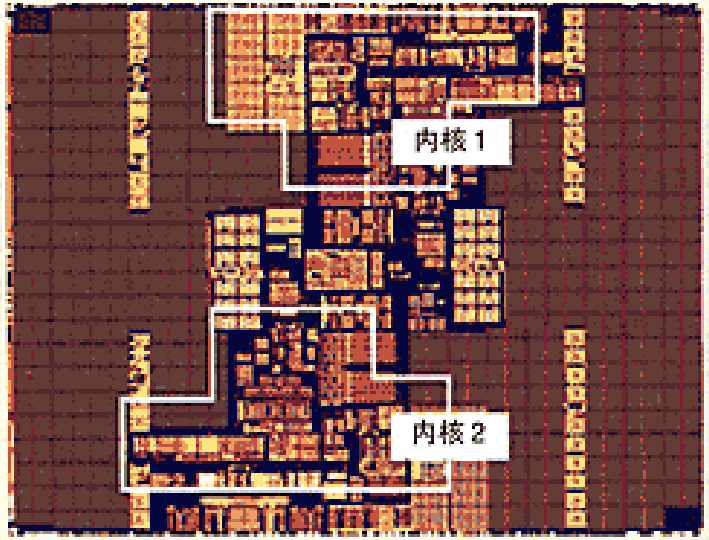
- 提高CPU运算速度途径
  - 提高主频，功耗、散热问题
  - 增加硬件处理部件，由并发处理变为并行处理
- CPU内核
  - CPU内部的运算器、微码存储器、高速缓存、寄存器和总线接口等硬件资源
- 双核CPU
  - 在一个CPU芯片内封装2个CPU内核，具备2套CPU计算资源
  - 2个CPU内核同时并行工作，运行进程/线程

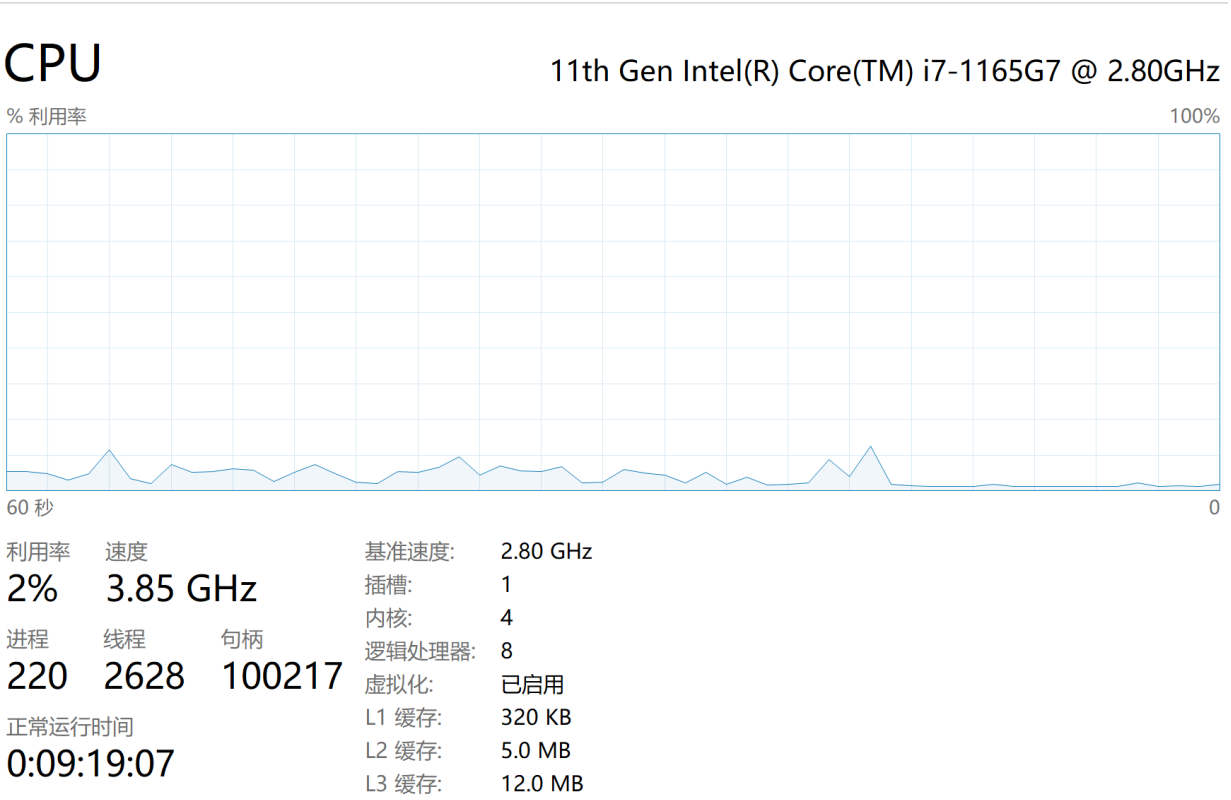
多核CPU架构

多内核架构

目前的通用多内核处理器设计都是基于对称多处理（SMP）技术的。SMP是一种并行架构，在这种架构中，所有的处理器运行一个操作系统副本，共享内存和一台计算机的其他资源，并且平等地访问内存、I/O及外部中断。大多数SMP架构是由下图演化而来的：

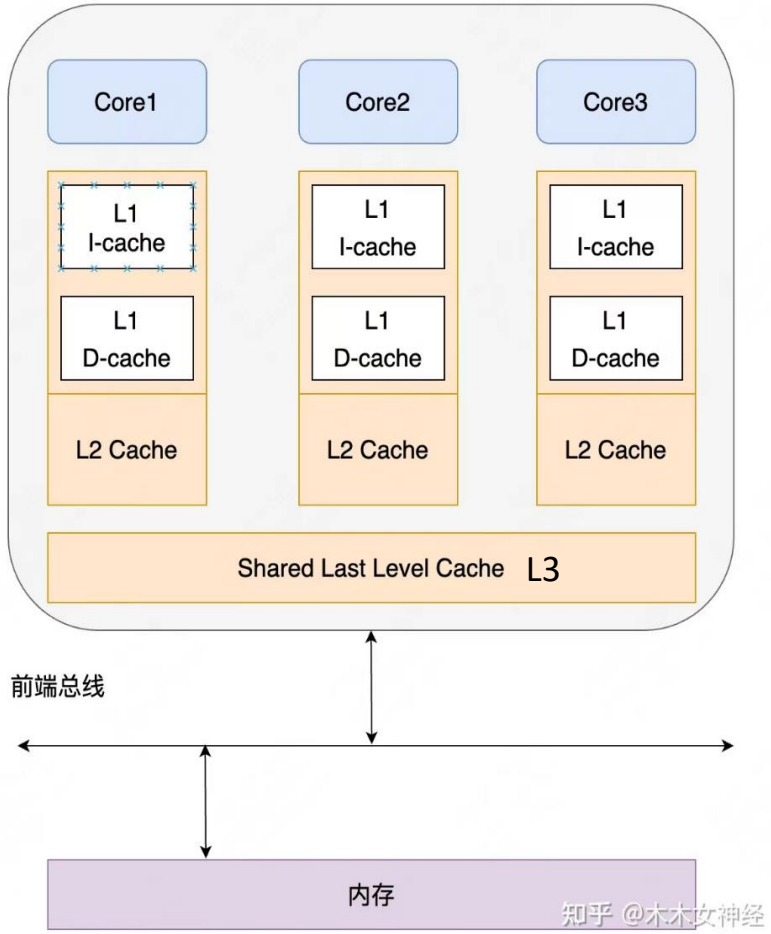






多级缓存cache

- 一级 Cache
  - 分为数据 Cache 和指令 Cache
  - 速度最快，数据访问速度3~5个指令周期
  - 成本高，容量小只有几十 至数百KB
  - 每个CPU核拥有自己的一级 Cache
- 二级 Cache
  - 数据和指令无差别地存放在一起
  - 速度比一级 Cache 慢，数据访问速度十几个指令周期
  - 容量较大，几百 KB 到几 MB 不等
  - 每个CPU核拥有自己的二级 Cache
- 三级Cache
  - 速度更慢，数据访问速度几十个指令周期
  - 容量大，几 MB 到几十 MB
  - 为多核CPU内部所有核所共有

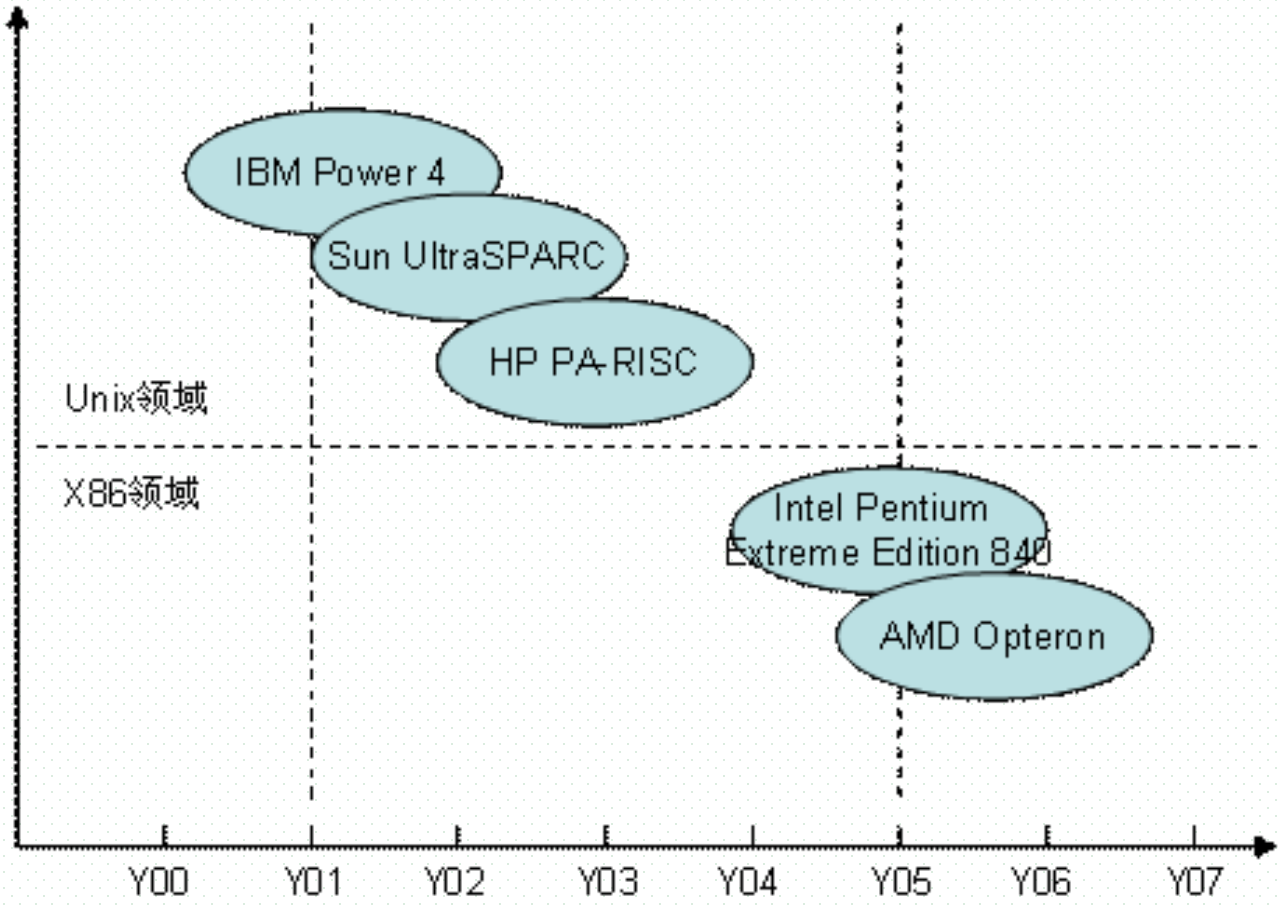


11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz

利用率	速度		基准速度:	2.80 GHz
7%	4.09 GHz		插槽:	1
			内核:	4
进程	线程	句柄	逻辑处理器:	8
215	2907	89916	虚拟化:	已启用
			L1 缓存:	320 KB
正常运行时间			L2 缓存:	5.0 MB
0:16:39:42			L3 缓存:	12.0 MB

3. 双核/多核CPU技术的发展

- 从Unix领域（服务器、工作站、主机）跨越到x86领域（微机）
- 进入x86领域之前，一直小范围应用于企业服务器领域
- Intel 2004年4月先后发布了两款针对桌面应用的双核处理器Pentium Extreme Edition 840和Pentium D处理器，同年AMD发布Opteron双核处理

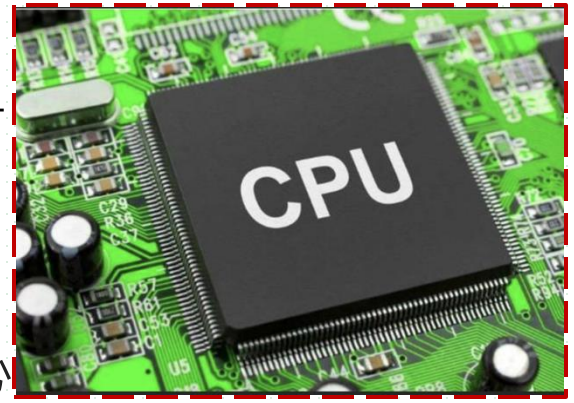


4. 双核/多核CPU应用

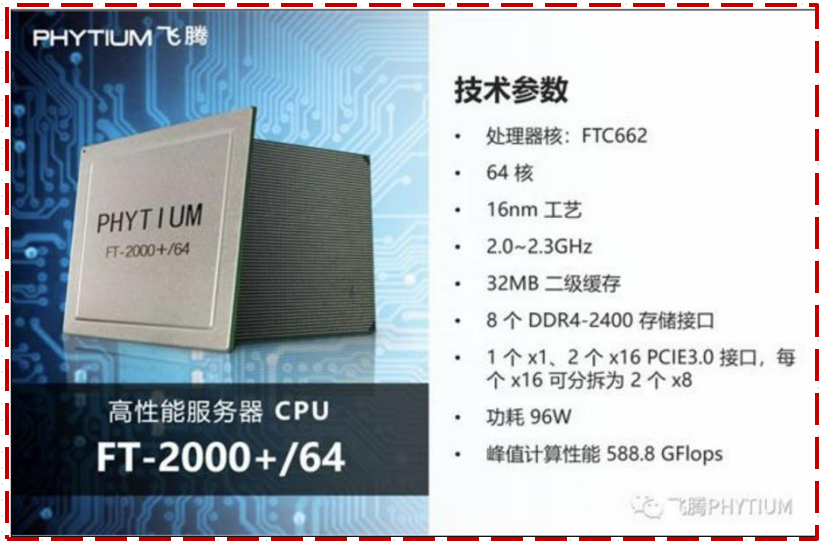
- 真正实现双核CPU系统的计算能力需要CPU芯片、主板芯片组、操作系统、编译系统、系统软件、应用软件等方面相互协同配合，
  - 编译系统需要将应用程序编译为可以并行执行的、多线程目标代码，例如OpenMP
  - 支持多核技术的虚拟机软件
  - 算法设计采用分治设计策略/方法
  - 多核多线程编程
    - multithreaded programming
- 优势
  - 散热量小
  - 运算速度快，双核系统平均较单核系统快60-70%
- 应用于计算密集型 (computation-intensive) 任务
  - 大型科学实验室、图形图像处理、高仿真视频模拟、气象预报、石油勘探、水利计算、生物信息等

5. 国产多核/众核CPU

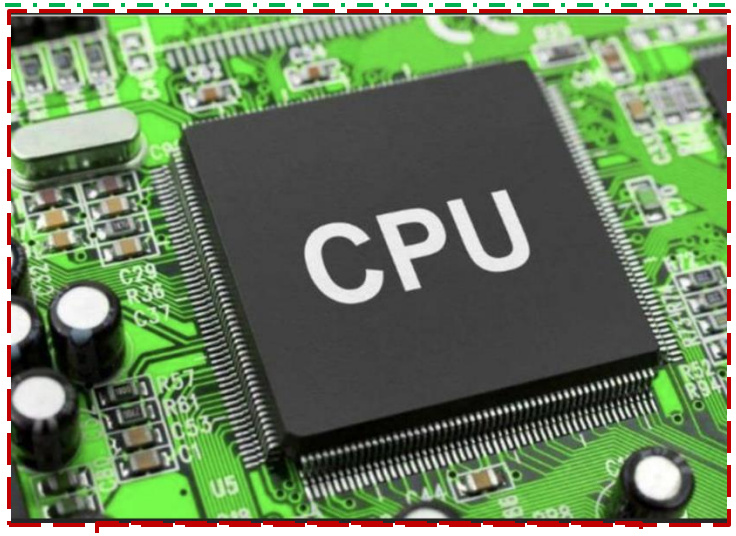
- 申威26010众核处理器，SW26010
  - 基于Alpha架构，260个运算核心；频率1.45GHz，8G DDR3，25平方厘米芯片
  - 每秒3万+亿次计算能力，3.06TFLOPS
  - 申威网络交换芯片
  - 神威太湖之光高性能计算机，装有40,960个芯片，260*40960=10649600个核心
  - 神威E级超算原型机



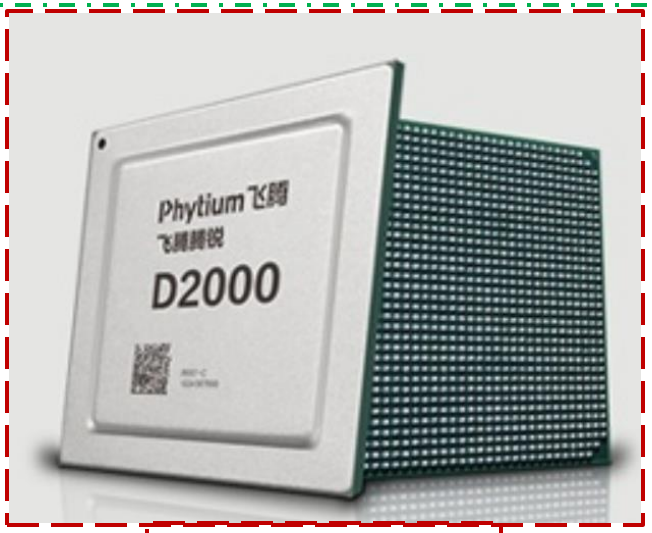
- 飞腾64核CPU芯片，FT-2000/64 (2016-08)
  - 64 核通用处理器，基于ARM 架构
  - 中国天津飞腾信息技术有限公司设计
  - 核心频率 2.0 G，浮点运算峰值速度每秒 5120 亿次，典型应用情形下的实测功耗 100 瓦
  - 长宽各为 55 毫米的封装内，借28 纳米集成电路工艺，集成 48 亿个晶体管
  - 用于天河超级计算机，作为管理结点，4096个？



龙芯：
MIPS架构，loongarch指令集



申威：
Alpha架构，sw64指令集



飞腾：
Arm架构/指令集

中科曙光~海光:
x86架构，AMD Zen1微架构

魔都~兆芯:
x86架构/ip内核/指令集

华为~鲲鹏：
ARM架构/指令集

阿里RV~玄铁：
基于开源RISC-V

Intel多核CPU

- Intel 13代酷睿CPU
  - 混合架构：性能核 + 效率核；7纳米工艺，DDR5
  - 性能核，P 核，大核
    - 支持超线程HT，频率更高
    - 提高单线程性能和响应速度
  - 效率核，E 核，小核
    - 无超线程HT，缓存cache等有删减，功耗低
    - 为多任务处理提供可扩展的多线程性能和高效的后台任务卸载
  - 通过P核、E核，提供均衡的单线程和多线程实践性能
- Thread Director（硬件线程调度器）
  - 辅助OS调度程序将工作负载智能分配到最佳内核，以优化工作负载
  - 最新版本 Windows11 22H2 内置了更高效的线程调度
  - 示例：在前台渲染视频时，默认采用高性能 P 核，以提供更快速度；但如果在渲染时打开 PS 制作图片，此时系统对多线程要求明显提高。此时，Thread Director 自动将渲染任务移动到高效能的 E 核，将 P 核让给前台的 PS，保证前台工作的稳定流畅

芯片面积：
1个大核=4个小核

Intel 13代酷睿多核CPU

For Gaming Laptops and Mobile Workstations

13th Gen Intel Core™ HX Processors

	Processor Number	Processor Cores	Processor Threads	Performance Cores	Efficient Cores	L3 Cache	Max Turbo Frequency P-cores	Max Turbo Frequency E-cores	Base Frequency P-cores	Base Frequency E-cores	Processor Graphics	Max Graphics Frequency	DDR5 Frequency	Processor Base Power	Max Turbo Power
intel CORE i9	i9-13980HX	24C	32T	8P	16E	36 MB	5.6 GHz	4.0 GHz	2.2 GHz	1.6 GHz	32 EU	1.65 GHz	5600	55 W	157 W
	i9-13950HX	24C	32T	8P	16E	36 MB	5.5 GHz	4.0 GHz	2.2 GHz	1.6 GHz	32 EU	1.65 GHz	5600	55 W	157 W
	i9-13900HX	24C	32T	8P	16E	36 MB	5.4 GHz	3.9 GHz	2.2 GHz	1.6 GHz	32 EU	1.65 GHz	5600	55 W	157 W
intel CORE i7	i7-13850HX	20C	28T	8P	12E	30 MB	5.3 GHz	3.8 GHz	2.1 GHz	1.5 GHz	32 EU	1.60 GHz	5600	55 W	157 W
	i7-13700HX	16C	24T	8P	8E	30 MB	5.0 GHz	3.7 GHz	2.1 GHz	1.5 GHz	32 EU	1.55 GHz	4800	55 W	157 W
	i7-13650HX	14C	20T	6P	8E	24 MB	4.9 GHz	3.6 GHz	2.6 GHz	1.9 GHz	16 EU	1.55 GHz	4800	55 W	157 W

32MB L2
36MB L3

CPU核数目

CPU线程数目 =性能核数*2 + 效率核数

性能核数目

效率核数目

P核频率高于E核频率

Appendix 4-3 多核/HT CPU速度提升(Speedup)

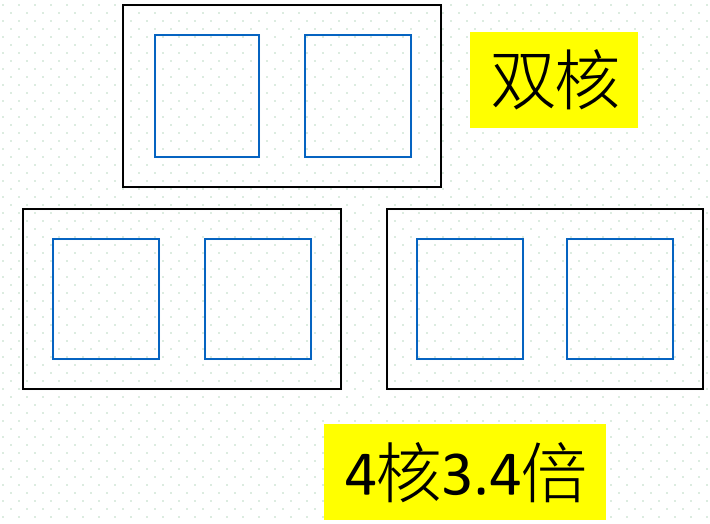
- Speedup, SP，并行加速比
  - 多处理器系统相比于单处理器系统的运算速度比，或
  - 多核CPU相比于单核CPU的运算速度比

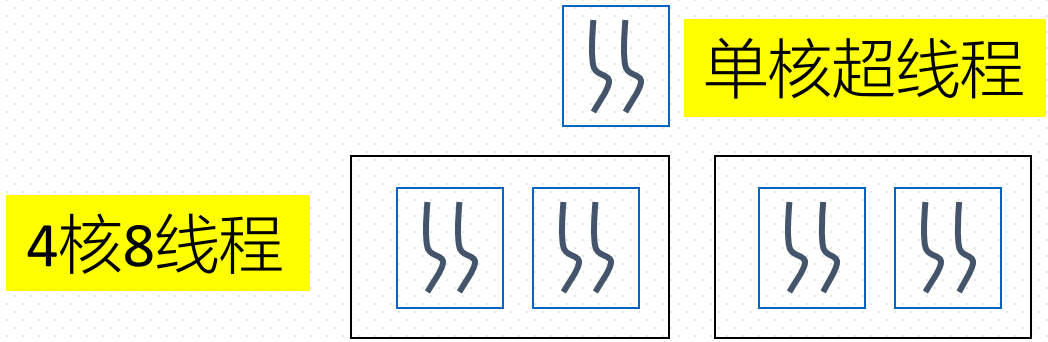
- 与常规单核、无超线程技术的CPU相比，n核、m线程CPU的性能大约提升到？倍
  - Intel酷睿i7 3770K，n=4核，m=2*n=8线程
  - Intel酷睿i9 139900K，n=8核，m=16线程
  - Intel酷睿i7 9700K，n=8核，m=8线程
  - Intel酷睿i9 13900K，n=(8P+16E)核，m=2*8+16=32线程

方法1. n核m线程（保守估计）

SP=[1.7 *(n/2)] * 1.3

- Step1. 双核性能提升
  - 2核较之单核大约快70%，
    n/2=2/2对CPU物理核的速度约为单核速度的
    =1.7 * n/2=1.7*2=3.4 倍
    - 加速比SP， speedup
- Step2. HT性能提升
  - 单核CPU采用超线程可提升性能30%，
    n=4核、 m=8超线程CPU的性能提升到：
    SP=3.4 * (1 + 30%) = **4.42**倍

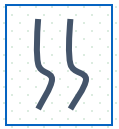




方法2. n核m线程（乐观估计）

SP=1.3n

- Step1. HT性能提升
  - 单核CPU采用HT可提升性能30%，
    1个物理核、2个逻辑核的性能为1.3



单核超线程1.3

- Step2. 多核性能提升
  - n=4个带有HT物理核的速度约为单核速度的

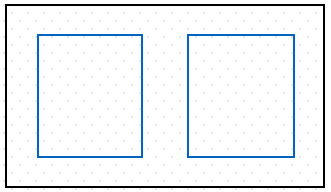
SP=1.3 * n=1.3*4=5.2倍

4核8线程

方法3. n核n线程无HT

SP=(1.7~1.9)*(n/2)

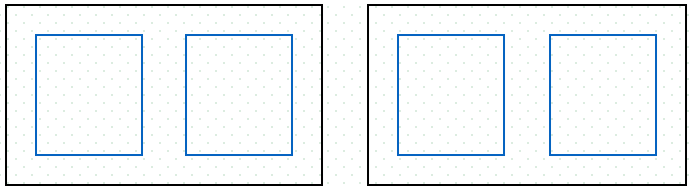
- Step1. 一对物理核性能由2降为1.7，由于内存访问冲突、资源竞争等



双核1.7

- Step2. n核CPU有n/2对物理核，多对物理核提升
  - n=4个物理核的速度约为单核

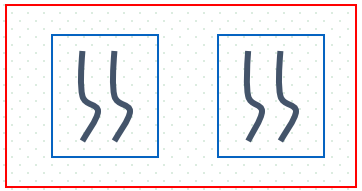
1.7 * n/2=1.7*2=3.4倍



4核4线程3.4倍

方法4. n核m线程（带HT，非线性乐观估计）

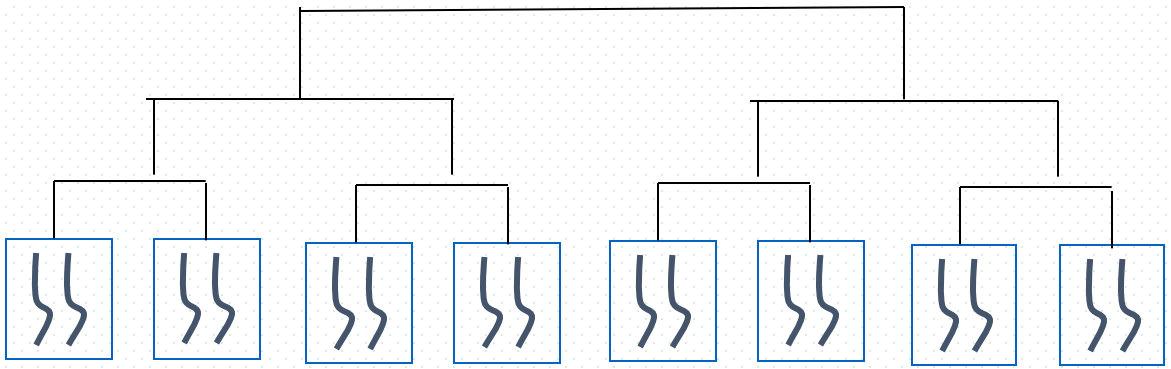
- Step1. HT性能提升1→1.3，双核性能降为2 → 1.7
  - 双核4线程性能



2核4线程1.3*1.7=2.21

- Step2. 多核性能提升
  - n=8 =2^3时，

    SP= $(1.7*1.3)^3$ = 10.79



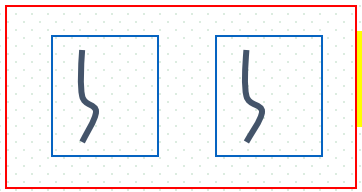
SP= $\lfloor (1.7 * 1.3)^{log_2 n} \rfloor$ (向下取整)

或: $(1.7 * 1.3)^{\lfloor log_2 n \rfloor}$

n	m	SP
2	4	2.21
4	8	4.88
8	16	10.79
16	32	23.85
32	64	52.64

方法5. n核n线程（无HT，非线性保守估计）

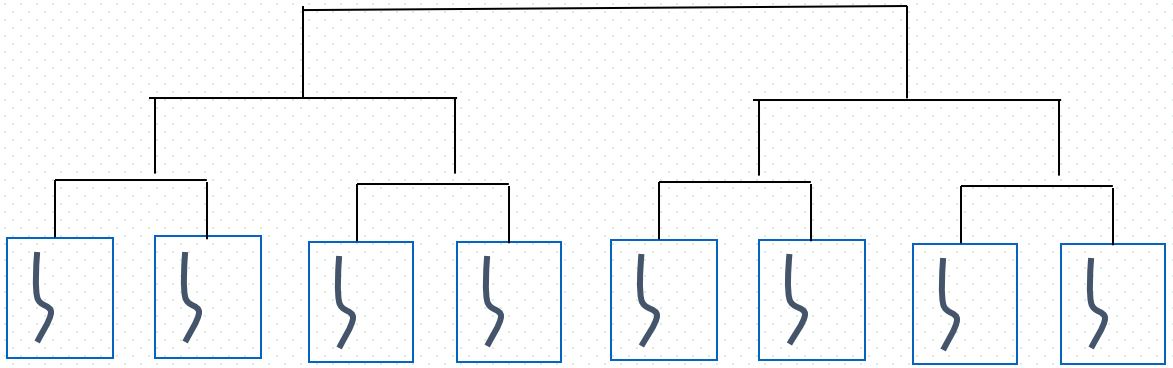
- Step1. 双核性能降为2 → 1.7，或1.8、1.9
  - 双核2线程性能



2核2线程1.7

- Step2. 多核性能提升
  - n=8 =23时，

    SP= $(1.7)^3$ = 4.9



SP= $\lfloor (1.7)^{log_2 n} \rfloor$ (向下取整)

或: $(1.7)^{\lfloor log_2 n \rfloor}$

n	m	SP
2	2	1.7
4	4	2.89
8	8	4.91
16	16	8.35
32	32	14.20

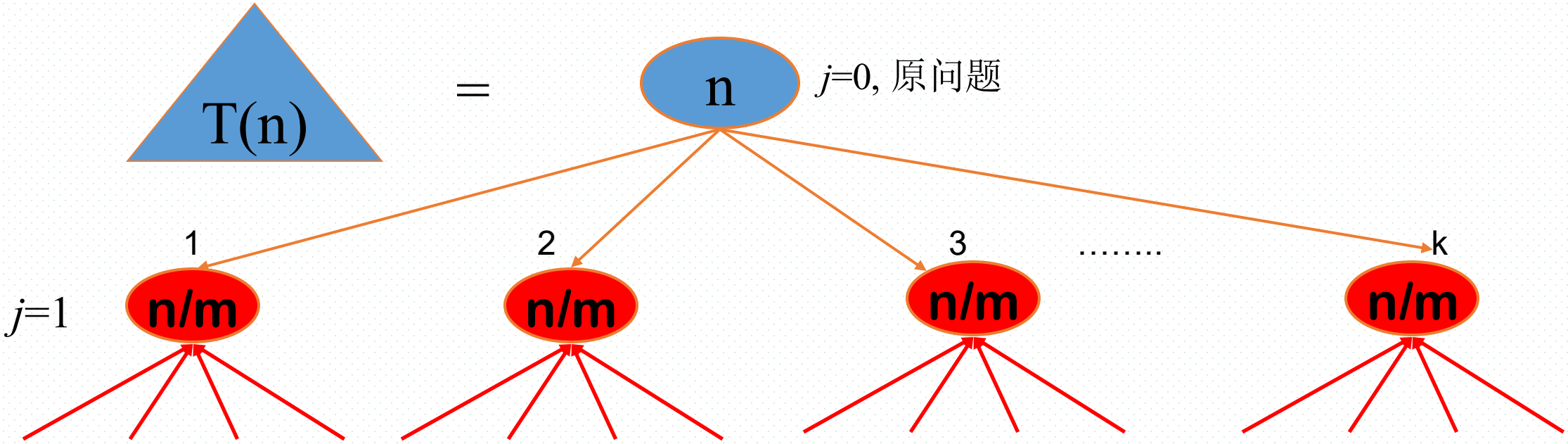
CPU亲和 (affinity)/绑定(banding)

- CPU 亲和力分为
  - 软亲和力
  - 硬亲和力
- Linux 内核进程调度器天生具有CPU 软亲和力
  - 多处理器系统中，进程通常不会在处理器之间频繁迁移
  - 进程迁移的频率小就意味着产生的管理负载小，但不代表不会进行小范围的迁移

- CPU 硬亲和力
  - 进程固定/绑定在某个处理器上运行，不在不同的处理器之间进行频繁的迁移
  - 减少CPU调度成本，改善程序性能

Appendix4-4 基于分治的并行问题求解及算法复杂性分析

- 本学期，算法设计与分析，分治算法设计策略
  - 将规模为n的问题分成*k*个规模为n/m的子问题，并行、串行求解子问题



原问题/子问题所处层次: j, j=0,1,2,…, $log_m n$-1
第j层原问题/子问题的规模: n/m^j
第j层原问题/子问题的总数: k^j

基于分治的并行问题求解

- step1. 任务分解

  复杂任务P分解为n个子任务，耗时T_1

  $P \rightarrow \{p_1, p_2, \ldots, p_n\}$

- step2. 子任务分派

  将$p_1, p_2, \ldots, p_n$分派到n个处理器上，耗时T_2

- step3. 子任务并行求解

  $p_1, p_2, \ldots, p_n$同时/并行求解，耗时T_3

- step4. 结果合成

  根据子问题$p_1, p_2, \ldots, p_n$的解，得到原问题P的解耗时T_4

总时间：$T_1 + T_2 + T_3 + T_4$

增大CPU数目n，降低T_3

设分解阀值$n_0=1$，且解规模为1的问题耗费1个单位时间。

再设将原问题分解为k个子问题以及用merge将k个子问题的解合并为原问题的解需用f(n)个单位时间。

用T(n)表示该分治法解规模为|P|=n的问题所需的计算时间，则有：

$$T(n)=\begin{cases} O(1) & n=1 \\ kT(n/m)+f(n) & n>1 \end{cases}$$

通过迭代法求得方程的解：

$$T(n)=n^{\log_m k}+\sum_{j=0}^{\log_m n-1} k^j f(n/m^j)$$

第j层子问题规模

K个子问题串行求解代价

各层原问题/子问题分解、合并代价累加

第j层子问题总数

第j层的1个子问题分解合并代价

并行算法加速比与阿姆达尔定律

- 并行算法加速比(speedup)
  - 同一个任务在单处理器系统和并行处理器系统上的运行时间的比率
  - 衡量并行系统或程序并行化的性能和效果

$$SP = T_1 / T_p$$

  - T_1:单处理器下的运行时间
  - T_p:有P个处理器并行系统中的运行时间

问题本身/程序结构对速度提升的影响?
对可并行求解的问题，并行计算速度提升的上限

- 当SP=P时，称为线性加速比(linear speedup)

阿姆达尔定律

- 并行算法加速比(speedup)公式
- 阿姆达尔(Amdahl)
  - IBM360系列机的主要设计者
- 阿姆达尔定律
  - 系统中对某一部件采用更快执行方式（e.g. 并行处理）所能获得的系统性能改进程度，取决于这种执行方式被使用的频率，或所占总执行时间的比例

- 该定律定义了采取增强（加速）某部分功能处理的措施后，可获得的性能改进或执行时间的加速比

- 结论
  - 通过更快的处理器所获得计算加速，受限于慢的系统组件

阿姆达尔定律

- 在固定负载情况下，加速比SP

$$sp = \frac{1}{\alpha + (1-\alpha)/n}$$

  - α : 串行计算部分所占比例，e.g. f(n), 子问题分解、任务分派解合并
  - n: 并行处理结点个数

- 当α =0时，可完全并行化，最大加速比sp=n；
- 当α =1时，无法并行化，最小加速比sp=1；
- 当n→∞时，分解后的子问题数目、并行计算节点数目趋于无穷大，
  - 极限加速比sp→ 1/α——加速比的上限

阿姆达尔定律：计算示例

- 多核系统中计算机游戏图形渲染
  - 加速比sp=19，处理器数目n=32，

$$sp = \frac{1}{\alpha + (1-\alpha)/n}$$

- 有
  - 19*(32α + (1- α)) = 32，　19*(31α + 1) = 32，
  - α = 0.022
    = 2.2%

- 加速比sp上限 = 1/α = 45

Thanks for your attention

北京郵電大學